PRINCIPAL PRODUCT FORENSIC AUDIT

LABOUR-PARTY-2

Flutter Reference Application — Forensic Audit & Native Android Specification

Repository: https://github.com/Manash07Bhoi/LABOUR-PARTY-2.git
Commit audited: 2dd2fe4ed85e9f0e2a420e3a383fed1e75b8a21b
Branch: main (audit on audit/documentation)
Audit date: 2026-09-07
Current implementation: Flutter / Dart (offline-first, single-user)
Future implementation: Native Android / Kotlin / Jetpack Compose / Material 3 / Firebase

This PDF is generated from the actual Markdown documentation in `docs/`. No native Android application has been implemented; native specifications are PROPOSED.

Table of Contents

EXECUTIVE SUMMARY

00-AUDIT-INDEX.md

Audit Index & Executive Control Document

Purpose: single executive roll-up of the forensic audit. Every claim is classified. Commit `2dd2fe4`. Audit date 2026-09-07.

1. Repository snapshot

Field	Value
Repository	`https://github.com/Manash07Bhoi/LABOUR-PARTY-2.git`
Commit audited	`2dd2fe4ed85e9f0e2a420e3a383fed1e75b8a21b`
Branch (audited HEAD)	`main` (remote `origin/main`); audit on `audit/documentation`
Tags	`RC-1.3`, `v1.0.0`, `v1.0.0-rc1`, `v1.0.1-hotfix-rc1`
Working tree	clean at clone
Contributors (git)	google-labs-jules[bot] (64), Manash07Bhoi (48); 112 commits
Dart files (lib)	47 (incl. 5 generated `.g.dart`); 6,140 lines
Test files	27; ~2,505 lines
Backend services	NONE (offline app)
Build config	AGP 8.11.1, Kotlin 2.2.20, Gradle 8.14, Java 17
Android	namespace/appid `com.roshan.labourparty`; minSdk 24; compile/target via Flutter defaults; version 1.0.0+1
Firebase	NONE present
CI/CD	NONE present

2. Audit environment & execution

- Environment: sandbox without a Flutter/Dart SDK (git, Python, Node, Java 11 present).
- Execution attempted: **not possible** — no Flutter SDK to install/build/test; no Android emulator. Audit is **static + evidence-based**.
- Runtime claims in prior repo docs (screenshots under `docs/screenshots`) were **not independently reproduced**. All runtime-dependent statements are `UNVERIFIED`.

3. Application status summary

Metric	Value	Classification
Product	Offline-first single-user labour/trip manager	VERIFIED
Screens (routes)	9 routed screens + splash (S-01..S-09)	VERIFIED

Metric	Value	Classification
Bottom-nav destinations	4 (Dashboard/History/Analytics/Settings)	VERIFIED
Features catalogued	29 (F-01..F-29); 17 working, 5 partial/stub, 7 missing/proposed	VERIFIED
Roles	0 (single user)	VERIFIED
Auth	None	VERIFIED
Backend	None	VERIFIED
Dependencies	flutter_bloc/go_router/hive/get_it/equatable/dartz/uuid/intl/screenutil/file_picker/google_fonts/flutter_animate/glassmorphism_ui/path_provider (runtime); build_runner/hive_generator/mocktail/flutter_lints (dev)	VERIFIED
Local data	Hive boxes: work/trip/labour/trip_labour/draft	VERIFIED

4. Critical / high / medium findings

Sev	ID	Finding
CRITICAL	SEC-1 / BUG-01 / SA-1	`android/app/keystore.jks.bak` (DER/PKCS#8 v0 private-key structure) committed on main + all 4 tags + ~26/31 remote branches; `.gitignore` misses `*.jks.bak`. Rotate & purge required.
HIGH	A-1/A-2	Accessibility: icon buttons untagged; low-alpha text contrast risk (native & current).
MEDIUM	BUG-02..05,09..13	Inert search/filter & orphan `/details`; filter unimplemented; misleading swipe; data-integrity/restore-auth; labour edit refresh; duplicate code.
MEDIUM	ST-1/2/15	PII at rest & backups unencrypted; backup unsigned.
MEDIUM	T-1	Test/execution not reproduced (no SDK in env).
LOW	BUG-06..08,14,16..19	Blank empty-state CTA; inert settings tiles; session-boundary doc drift; no retry; dead code; unused phone field.

5. Ratings (qualitative, evidence-based — NOT fabricated numeric scores)

Dimension	Rating	Basis
PRODUCT MATURITY	MODERATE (single-user core is coherent; multi-role/backend absent)	verified scope
ENGINEERING MATURITY	MODERATE (clean-ish layering but presentation leaks, dead code, no CI/migration framework)	code audit
SECURITY MATURITY	WEAK-FOR-MULTIUSER / LOW-RISK-TODAY; one CRITICAL config issue (keystore)	SECURITY-AUDIT
UX MATURITY	MODERATE (polished core; inert/dead paths; accessibility gaps)	UX-AUDIT
TESTING MATURITY	MODERATE (good test inventory but execution unverified + coverage gaps)	TESTING-AUDIT
PERFORMANCE MATURITY	UNVERIFIED (structural design good; no runtime measurement)	PERFORMANCE-AUDIT
PRODUCTION READINESS	**NOT READY / UNVERIFIED** — secret-hygiene FAIL (keystore) blocks any release; runtime gate not run	PRE-RELEASE

Dimension	Rating	Basis
NATIVE MIGRATION READINESS	READY TO SPEC, core domain low-risk; auth/backend/sync are net-new	FEATURE-PARITY-MATRIX

6. Documentation index

See `DOCUMENTATION-INDEX.md` (full tree). Required docs verified present (programmatic check in `verify` section).

7. Migration & production readiness statements

- **Native migration readiness:** Core offline domain (F-01..F-19) maps cleanly and is low-medium risk to rebuild. The secure multi-user backend, roles, notifications and sync (F-23..F-27) are new high-risk scope. Start with Phase 0 decisions + keystore remediation (IMPLEMENTATION-ROADMAP.md).
- **Production readiness (current Flutter):** NOT READY for an actual public release until (1) keystore file removed/rotated, (2) the release gate is executed on a Flutter-enabled machine, (3) data-integrity/migration tests are re-run. Functionally coherent, but readiness is UNVERIFIED in this environment.

8. Unresolved questions (UNVERIFIED)

OQ-1 operator count · OQ-2 owner identity/brand ("Ramesh Sahu" vs package "roshan") · OQ-3 cloud vs offline-only future · OQ-4 exact session boundary (code vs PRD text) · OQ-5 soft vs hard delete future · OQ-6 audit/retention depth · OQ-7 driver/labourer login scope · OQ-8 audit granularity. Runtime numeric validation for many claims is UNVERIFIED.

9. Overall audit conclusion

The existing product is a **genuinely useful, coherent offline-first single-user labour/trip tracker** with a well-scoped domain and good structural intent. It is **not** a multi-user/RBAC product, **not** connected to any backend, and it carries **one critical repository-hygiene/security issue** (committed keystore-like file). The documentation here separates current reality from proposals so the native rebuild can preserve validated behaviour and deliberately fix the weak areas.

Final Audit Report

Repository

- Commit audited: `2dd2fe4ed85e9f0e2a420e3a383fed1e75b8a21b`
- Branch: `main` (work on `audit/documentation`)
- Build status: NOT BUILDABLE in this environment (no Flutter SDK). Static structure is release-shaped; execution UNVERIFIED.

Product

- Purpose: offline-first labour/trip management (Work→Trip→Labour+attendance), history, analytics, backup/restore.
- Users: single operator. Roles: none.
- Major features: F-01..F-19 working core; F-20..22 partial; F-23..29 absent/proposed.

UX

- Screens: S-01..S-09 (9 routed + splash). Navigation: splash→shell(4 tabs); pushed detail/edit/confirm screens.
- Major UX problems: orphan `/details`, inert search/filter, misleading swipe delete, no retry, search only today, accessibility, dark-only.
- Opportunities: role IA, day detail, history search/filter, bottom-sheet filters, light/dynamic theme.

Engineering

- Architecture: Flutter + flutter_bloc + get_it + go_router + Hive + dartz; Clean-ish layering w/ presentation leaks.
- Dependencies/data/APIs: listed above; no remote API; Hive schema; BLoC state; type failures.

Security

- Critical: committed keystore-like `.jks.bak` (main+tags+branches).
- High: (accessibility aside) none; current runtime attack surface minimal offline.
- Medium: PII at rest/backup unencrypted; backup unsigned; restore trusts content.
- Low: raw-exception text; duplicate-code merge hygiene; deep-cast nav.

Performance

- Measured: none re-run. Suspected risks: full-box scans; labour name lookups; backup memory; fonts network fetch.
- Unverified: all numeric (cold start, jank, benchmarks).

Testing

- Existing: 27 files (unit/widget/integration/perf/e2e + mock helper). Missing: execution confirmation; settings/details/draft/confirm UI tests; accessibility/security/locale tests.
- Execution results: UNVERIFIED (no SDK).

Firebase

- Existing: none. Recommended: Auth/Firestore/Storage/CF/AppCheck/Crashlytics (+FCM, optional Analytics/Remote Config), server-authoritative rules & audit (08-NATIVE-ANDROID).

Native Android

- Recommended architecture: Kotlin+Compose+M3+Navigation+ViewModel+StateFlow+CleanArch+Hilt+Coroutines+Repository+Room/DataStore/Coil/WorkManager+Firebase.
- Migration complexity: domain low-medium; UI medium; auth/backend/sync high.
- Redesign opportunities: role IA, day detail, filters, accessibility, theme, backup/DR.

Production readiness

NOT READY for public release (secret hygiene fails; release gate unexecuted). Functionally coherent but readiness **UNVERIFIED** here. Recommend: remediate keystore, run the gate on a Flutter-enabled CI, then reassess.

Native Android migration readiness

READY TO SPEC / LOW-MED CORE RISK — proceed to Phase 0 (decisions + keystore remediation) then native implementation.

Known unverified areas

Flutter build/test/run · benchmark numbers · runtime UX/screenshots · TalkBack/contrast numeric · CVE status of deps · whether a Play-store release exists · deeper per-branch secret history.

PRODUCT SPECIFICATION

01-PRODUCT/PRD.md

PRD — Labour Party (Flutter reference implementation audit)

0. Document metadata

Field	Value
Purpose	Authoritative product requirements document reconstructed from the existing Flutter repository
Scope	Existing product behaviour only; future native Android requirements are separately labelled `PROPOSED`
Status	**VERIFIED (from repository & evidence)** where stated; gaps marked `UNVERIFIED` / `PROPOSED`
Evidence/ source	`README.md` , `PRD.md` (root), `lib/**` , `docs/PRODUCT_OVERVIEW.md` , `pubspec.yaml`
Repository	`https://github.com/Manash07Bhoi/LABOUR-PARTY-2.git`
Commit audited	`2dd2fe4ed85e9f0e2a420e3a383fed1e75b8a21b`
Audit date	2026-09-07
Classification guide	`VERIFIED` = observed in code; `INFERRED` = strongly supported but not directly observed; `PROPOSED` = future recommendation; `UNVERIFIED` = could not be confirmed

> **IMPORTANT SCOPE NOTE (VERIFIED).** The existing product is an **offline-only, single-user, local-first** Android application. It has **no authentication, no user accounts, no roles, no backend, no network, and no cloud sync.** The owner/driver/laborer role model and Firebase backend requested in the migration brief are **future objectives, not current functionality.** They are documented as `PROPOSED` and are detailed in `08-NATIVE-ANDROID/` and `04-SECURITY/RBAC.md`. Do not confuse these categories.

---

1. Executive summary

Labour Party is an offline-first labour/trip management application for Android. It lets an operator (the app owner) record a day's work broken into **Work → Trips → Labour** records, track tractors and drivers per trip, mark labourer attendance per trip, browse history by date/session, view basic analytics, and export/import the local dataset as a `.labourbackup` file.

The product is explicitly scoped (per the root `PRD.md`) to: **No login, No internet, No cloud sync, No multi-user, No external backend.** All data lives inside Hive NoSQL boxes in the Android application sandbox.

2. Product vision

`INFERRED` (from code, README, and existing docs): A fast, reliable, 100 % offline tool for a single operator to run and record a labour/transport day (sand, soil, stone haulage with tractor trips) without depending on connectivity, cloud services, or per-user accounts.

3. Problem statement

`INFERRED`: Manual record keeping of daily tractor trips and daily-wage labour attendance is error-prone and hard to reconstruct. The app digitises the ledger on-device with strong data-integrity and trip-numbering rules and a backup file so records are not lost when the device is changed.

4. Objectives

ID	Objective	Category
OBJ-1	Record a `Work` (date + session + work type + place)	VERIFIED
OBJ-2	Record many `Trip` records under a work (tractor, driver, trip number, time)	VERIFIED
OBJ-3	Track labour participation & attendance per trip (`isPresent`)	VERIFIED
OBJ-4	Maintain correct sequential trip numbering across morning/evening sessions and across dates	VERIFIED
OBJ-5	Preserve data integrity on delete/edit/restore (no orphaned records)	VERIFIED (intent; partial gaps)
OBJ-6	Provide history browsing grouped by date/session	VERIFIED
OBJ-7	Provide lightweight analytics (counts, driver ranking, tractor usage table)	VERIFIED
OBJ-8	Export/import local data via `.labourbackup`	VERIFIED
OBJ-9	Autosave in-progress form as a draft to prevent data loss	VERIFIED
OBJ-10	Operate fully offline	VERIFIED

5. Non-objectives (VERIFIED — current product)

- No login / registration / password recovery.
- No multi-user or role-based access control.
- No cloud backend / Firebase / remote APIs / sync.
- No online analytics or crash reporting (external).
- No payments, reports-as-service, notifications, announcements.
- No web presence / SEO.

These are **current** non-objectives. The migration brief proposes changing many of them for the native product (see `PROPOSED` sections).

6. Target users

Persona	Current (VERIFIED)	Future (PROPOSED)
App owner / operator (a labour-contractor / transport operator)	Only user of the app	Primary admin (brief names owner as "Ramesh Sahu" — NOT present in code)
Driver	Not a separate app user (driver is a data field on a Trip)	Potential role

Persona	Current (VERIFIED)	Future (PROPOSED)
Labourer / worker	Not an app user (labourer is an entity + attendance toggle)	Potential role

> The owner identity **"Ramesh Sahu" does not appear anywhere in the repository source** (grep `ramesh|sahu` returned no hits). The Android package/namespace id is `com.roshan.labourparty`. This is a documented discrepancy — see `03-ENGINEERING/AGENT.md` and `08-NATIVE-ANDROID/`.

7. Roles

Role	Current implementation	Access model
App user (implicit owner)	Unauthenticated; all features available	No auth gate (VERIFIED)
Owner / Admin	N/A — no auth, no role data	PROPOSED (backend-authoritative)
Driver	A `driverName` string on Trip; no account	PROPOSED
Labourer	A `Labour` entity + per-trip attendance; no account	PROPOSED

See `04-SECURITY/RBAC.md` for the RBAC matrix.

8. Core domain concepts (VERIFIED)

- **Work** — top-level record grouping a session of trips. Fields: `id, date, session, workType, place, createdAt, updatedAt` (`lib/features/work/domain/entities/work.dart`).
- **Trip** — one transport cycle. Fields: `id, workId, tripNumber, tractor, driverName, createdAt, place, workType, notes, updatedAt, status` (`.../entities/trip.dart`).
- **Labour** — a worker record. Fields: `id, name, phoneOptional?, createdAt` (`.../entities/labour.dart`).
- **TripLabour** — attendance join between Trip and Labour. Fields: `id, tripId, labourId, isPresent` (`.../entities/trip_labour.dart`).
- **Draft** — transient autosave of an in-progress new-trip form (`.../data/models/draft_model.dart`).

9. Use cases (primary)

UC	Description	Status
UC-1	Open app → splash → dashboard for today's date & auto-detected session	VERIFIED
UC-2	Add a work+trip via "Add Work"/"Add First Trip" form	VERIFIED
UC-3	Add the "next trip" by copying the last/latest trip & labours, editing, confirming	VERIFIED
UC-4	View today's trip list, search by driver/tractor/trip no./work type/place	VERIFIED (search)
UC-5	Open trip details; add/edit/remove labour; toggle attendance	VERIFIED
UC-6	Delete the latest trip or a specific trip (with confirm)	VERIFIED
UC-7	Browse history grouped by date→session	VERIFIED
UC-8	View analytics KPIs + sortable trip data table	VERIFIED
UC-9	Backup database to `.labourbackup`; restore with rollback	VERIFIED
UC-10	Resume an unsaved draft	VERIFIED

10. Functional requirements (FR) — current

> Requirement IDs are assigned by this audit for traceability. Status legend per `00-AUDIT-INDEX.md`.

FR-ID	Requirement	Implementation evidence	Status
FR-01	App launches to `/splash`, auto-navigates to `/dashboard` after ~1.5 s	`splash_screen.dart` `Future.delayed(...context.go('/dashboard'))`	VERIFIED
FR-02	Dashboard auto-loads current date and derived session	`dashboard_screen.dart` `initState`; `DateTimeUtils.getCurrentSession()`	VERIFIED
FR-03	Session derivation: hour 4–11 → Morning else Evening	`core/utils/date_time_utils.dart`	VERIFIED
FR-04	Add new Work+Trip with validation (work type, driver required; ≥1 labour)	`add_edit_work_screen.dart` `_saveWork`	VERIFIED
FR-05	Preserve date/session isolation when editing (throw if `editingTrip` without `editingWork`)	`add_edit_work_screen.dart` initState	VERIFIED
FR-06	Compute next trip number across morning+evening works for a date	`work_usecases.dart` `CalculateNextTripNumberUseCase`	VERIFIED
FR-07	Copy last trip's labours & place into a new "next trip"	`work_bloc.dart` `_onNavigateToConfirmNextTrip`	VERIFIED
FR-08	Save a new next trip and its labour attendances atomically-ish	`work_bloc.dart` `_onSaveNextTrip`; `_onSaveFullWorkTrip`	VERIFIED (see integrity notes)
FR-09	Search dashboard trips (driver, tractor, trip number, work type, place; case-insensitive)	`work_bloc.dart` `_onSearchDashboard` + `SearchDashboardEvent`	VERIFIED
FR-10	Filter dashboard	`FilterDashboardEvent` exists in state layer	**PARTIAL / UNREACHABLE** — no UI dispatches filter; filter predicate not implemented (always true)
FR-11	Delete latest trip / specific trip with confirmation	`_onRemoveLatestTrip`, `_onDeleteSpecificTrip`; dialogs	VERIFIED
FR-12	Trip detail: view info, add labour, edit labour name, toggle presence, remove labour (with Undo)	`trip_details_screen.dart`	VERIFIED
FR-13	History grouped by date & session, expandable; edit/delete/tap-through	`history_screen.dart`; `history_bloc.dart`	VERIFIED
FR-14	Analytics KPIs (works, trips, top driver) + sortable trip data table	`analytics_screen.dart`	VERIFIED
FR-15	Backup to `.labourbackup` (versioned JSON, ≤25 MB), SAF file save	`settings_screen.dart` `_backupDatabase`	VERIFIED

FR-ID	Requirement	Implementation evidence	Status
FR-16	Restore from `.labourbackup` with validation, pre-restore snapshot, rollback, count verification	`settings_screen.dart` `_restoreDatabase`	VERIFIED
FR-17	Autosave draft on field change / lifecycle pause; offer restore snackbar	`add_edit_work_screen.dart`	VERIFIED

11. Non-functional requirements (NFR) — current

NFR-ID	Requirement	Status
NFR-01	100 % offline operation; no internet permission in main manifest	VERIFIED
NFR-02	Dark-only Material 3 UI with glassmorphism & animation	VERIFIED
NFR-03	Data persists across restarts via Hive	VERIFIED
NFR-04	Cold start shows splash + skeleton loading	VERIFIED
NFR-05	Bulk `TripLabour` writes use `putAll`/prefix-key scans for performance	VERIFIED (see `05-PERFORMANCE/`)
NFR-06	No secrets/keys in source (client has no secrets)	PARTIAL — see `keystore.jks.bak` finding
NFR-07	Accessibility, localization, RTL, tablet layout	**MISSING / minimal** (see audits)

12. Business rules (VERIFIED, summary)

See `01-PRODUCT/BUSINESS-RULES.md` for the full register. Headline rules: - BR-1 Work is uniquely scoped by `(date, session)`; dashboard queries today's date/session. - BR-2 Trip number is derived: first=1, next=max(previous)+1 across morning & evening, edit preserves, delete preserves history, new date/session resets. - BR-3 Labour names persist exactly as typed ("never `Labour 1`"). - BR-4 A trip must belong to one date/session/work. - BR-5 Deleting a trip must cascade-delete its `TripLabour` attendance rows. - BR-6 Attendance `isPresent` must be stored per trip per labour.

13. Dependencies, assumptions, constraints

- **Dependencies**: `flutter_bloc`, `go_router`, `hive`/`hive_flutter`, `get_it`, `equatable`, `dartz`, `uuid`, `intl`, `flutter_screenutil`, `file_picker`, `google_fonts`, `flutter_animate`, `glassmorphism_ui`, `path_provider`, `cupertino_icons`. Dev: `flutter_lints`, `build_runner`, `hive_generator`, `mocktail`. (See `03-ENGINEERING/THIRD-PARTY-DEPENDENCIES.md`.)
- **Assumptions**: single device per install; operator trusts their own device; no multi-tenancy needed.
- **Constraints**: `minSdk 24`, Android only, offline, no backend. Package id `com.roshan.labourparty`.

14. Success metrics (PROPOSED for native; no telemetry currently)

Because the product is offline and has no analytics, **no runtime success metrics are measurable** (VERIFIED absence). Metrics below are PROPOSED targets for the native rebuild: - Data entry time per trip <

30 s; crash-free sessions ≥ 99.5 %; zero silent data-loss incidents; successful backup/restore round-trip 100 %.

15. Acceptance criteria (current, inferred from PRD & tests)

- A new day with no records shows the empty dashboard.
- Adding the first trip yields Trip #1; appending across sessions increments correctly (covered by `test/unit/usecases/trip_numbering_test.dart`).
- Editing a past trip preserves its `id`, `tripNumber`, `createdAt`, `date`, `session`.
- Restoring an invalid/large backup is rejected without destroying current data.
- Deleting a trip removes it from dashboard and history consistently.

16. Risks

Risk	Detail	Status
Committed keystore/private-key backup file	`android/app/keystore.jks.bak` tracked	**CRITICAL** — see `04-SECURITY/`
No authentication/RBAC	Any device holder with app can read/write all labour & phone data	HIGH (as product grows)
Local-only = single point of failure	No cloud backup	MEDIUM (by design today)
Backup file plaintext PII	Labour names + optional phone numbers exported unencrypted	MEDIUM
Duplicate top-level `LabourFormModel` declarations & duplicated switch cases	Code-maintainability/dead-code risk	MEDIUM/LOW

17. Future opportunities (PROPOSED)

- Backend-backed owner/driver/laborer roles with server-enforced RBAC.
- Optional Firebase sync/backup while retaining offline-first behaviour.
- Reports (daily wage tally, material totals), payments ledger.
- Notifications & announcements.
- Multi-device / company account structure.

18. Open questions

- OQ-1 Who operates the app in production and is a single-operator assumption correct? `UNVERIFIED`
- OQ-2 Intended owner identity (the brief's "Ramesh Sahu" vs package id "roshan"). `UNVERIFIED`
- OQ-3 Is any cloud/backend planned for the native product, or must it stay offline-only? `UNVERIFIED`

PRD2 — Operational & behavioural product specification

0. Purpose & relationship to PRD

PRD.md captures the product-level view. **PRD2.md** documents the deeper operational/behavioural specification: workflows, state transitions, validation, edge cases, failure/offline behaviour, data ownership, and audit requirements for the **current Flutter reference product**. Native-rebuild additions are `PROPOSED` and live mainly in `08-NATIVE-ANDROID/`.

- Status: VERIFIED (from code) unless labelled.
- Evidence: `lib/features/work/presentation/bloc/*`, screen files, `core/database/hive_setup.dart`, `settings_screen.dart`.

1. Global state machine (VERIFIED)

The UI is driven by BLoC states (`work_state.dart`, `history_state.dart`). Top-level states: `WorkInitial → WorkLoading → (DashboardLoaded | WorkEmpty | WorkError | TripDetailsLoaded | WorkActionSuccess | NavigateToConfirmNextTripState)`.

History states: `HistoryInitial → HistoryLoading → (HistoryLoaded | HistoryEmpty | HistoryError)`.

State transition table

From	Trigger	To
Any	`LoadDashboardDataEvent(date, session)`	`WorkLoading` → `DashboardLoaded` / `WorkEmpty` (if no work & no trips) / `WorkError`
DashboardLoaded	`NavigateToConfirmNextTripEvent`	`NavigateToConfirmNextTripState` then re-emits the stored dashboard state
Any	`SaveNextTripEvent` / `SaveFullWorkTripEvent`	`WorkLoading` → `WorkActionSuccess` → reload dashboard
DashboardLoaded	`DeleteSpecificTripEvent` / `RemoveLatestTripEvent`	`WorkLoading` → reload dashboard
Any	`LoadTripDetailsEvent(tripId)`	`WorkLoading` → `TripDetailsLoaded` / `WorkError`
Dashboard/ History	`LoadHistoryEvent`	`HistoryLoading` → `HistoryLoaded` / `HistoryEmpty` / `HistoryError`

> Finding (VERIFIED): `AddQuickTripEvent` handler `_onAddQuickTrip` is **empty** — the comment states navigation is performed directly in the UI. Dead event + empty handler (see `06-QUALITY/BUG-DEFECT-REGISTER.md`).

2. Session & date rules (VERIFIED)

- Date string format `dd MMM yyyy` (`DateFormat('dd MMM yyyy')`).
- Session: hour `4 ≤ h < 12` → `Morning`, else `Evening` (`getCurrentSession`). NOTE: this differs from the root `PRD.md` claim ("Morning 00:00–11:59"); **code says 04:00–11:59**, i.e. 00:00–03:59 is Evening. Discrepancy to resolve (`BUSINESS-RULES.md`).

- Data is queried/grouped by literal `date` string and `session` string; there is no timezone handling beyond `DateTime.now()` local time.

3. Workflow: Add a new work + first/next trip

Two overlapping entry flows exist (VERIFIED):

A. "Add Work" / "Add First Trip" (FAB & empty state) → `push('/add-edit-work', {isNew:true})` → `AddEditWorkScreen`. - If editing (from Details/History/Trip-details) it passes `editingTrip`, `editingWork`, `editingLabours`. - On Save: requires ≥1 labour; creates/overwrites Work with deterministic id `work_<date-with-underscores>_<session>`; creates a new Trip with `tripNumber=0` (resolved to next number in BLoC); saves labour master records + TripLabour attendance. On success emits `WorkActionSuccess` → snackbar → `context.pop()` → dashboard reloads.

B. "Next trip" quick add (plus button on Current-Trip card) → `NavigateToConfirmNextTripEvent` → dashboard listener copies last trip's place/workType/labours → `push('/confirm-next-trip', {...})` → `ConfirmNextTripScreen`. On Save emits `SaveNextTripEvent` → `WorkActionSuccess` → snackbar → pop → reload.

> **Concurrency/ `context.pop` finding (VERIFIED):** `ConfirmNextTripScreen._onSave` calls `context.pop()` immediately after adding the event, while the BLoC also reacts to `WorkActionSuccess`. `AddEditWorkScreen` relies on a `WorkActionSuccess` listener to pop. Because these are separate routes, double-pop is avoided only because each screen handles its own pop; but the ordering relies on BLoC listeners vs local pop and is fragile.

4. Workflow: Trip numbering (VERIFIED)

`CalculateNextTripNumberUseCase.call(date)`: 1. Fetch Morning work for date; if exists fetch its trips and compute max tripNumber. 2. Fetch Evening work for date; if exists compute max tripNumber. 3. Next = max(morning, evening) + 1. Evidence: `work_usecases.dart`, tested by `test/unit/usecases/trip_numbering_test.dart`.

5. Workflow: Labour & attendance (VERIFIED)

- Labour master list lives in `labour_box` (unique id per labour; name persists as typed).
- Per-trip attendance in `trip_labour_box` keyed by composite `"<tripId>_<tripLabourId>"`.
- Add labour in trip-details: creates a `Labour` master + a `TripLabour` with `isPresent=true`, then reloads trip details.
- Edit name: `SaveLabourEvent` updates the master record; comment notes it does not reload trip details and just emits `WorkActionSuccess` — **master name edits may not refresh the open trip list** (behavioural risk).
- Presence toggle: `UpdateTripLabourEvent` writes `isPresent` and reloads.
- Remove: `DeleteTripLabourEvent` hard-deletes the row; snackbar with **Undo** that re-saves (`SaveTripLabourEvent`).
- In `AddEditWorkScreen` the labour rows are in-memory (`LabourFormModel`) until saved.

6. Workflow: Delete a trip (VERIFIED)

- Delete triggers cascade of `TripLabour` rows whose composite key starts with `"<tripId>_"` (prefix scan) then deletes the Trip (`work_local_data_source.dart deleteTrip`).
- History deletion also refreshes the history list after a 300 ms delay.

7. Backup / restore workflow (VERIFIED)

Backup: 1. `FilePicker.platform.saveFile` with suggested name `labour_backup_<timestamp>.labourbackup`. 2. Serialise all four boxes to JSON `{version, createdAt, app:"Labour Party", data:{works, trips, labours, tripLabours}}`. 3. Write to the user-selected path (SAF).

Restore: 1. `FilePicker.platform.pickFiles`; require `.labourbackup` extension. 2. Off-main-thread parse+validate via `compute(_parseAndValidateBackup, path)`: file ≤ 25 MB; valid JSON map; `app=="Labour Party"`; `version` & `data` present; expected top-level arrays non-null; row-count upper bounds (works≤10000, trips≤100000, labours≤5000). 3. Show summary dialog (backup date, size, counts) with explicit overwrite warning. 4. Snapshot current boxes → clear boxes → write restore records → verify counts match → success snackbar. 5. On any error: roll back from the snapshot and show an error dialog.

8. Failure / error / edge-case behaviour (VERIFIED, summary)

Scenario	Behaviour	Evidence
Empty current work day	`WorkEmpty` → empty-state widget	`dashboard_screen.dart`
Repository/Db failure on load	`WorkError` with message text (no retry button on dashboard)	`dashboard_screen.dart`
Form missing work type/driver/labour	Inline validation + snackbar "At least one labour is required."	`add_edit_work_screen.dart`
`editingTrip` w/o `editingWork`	Throws `Exception` at initState	`add_edit_work_screen.dart`
Backup file invalid/too large/unsupported	Specific error dialogs	`settings_screen.dart`
Restore count mismatch / exception	Rollback from snapshot + error dialog	`settings_screen.dart`
No trips to show under a work	"No trips in current session" / "No trips found."	dashboard/details
History empty	Empty-state widget	`history_screen.dart`
Analytics empty	Empty-state widget	`analytics_screen.dart`

> **Gap (VERIFIED):** Dashboard `WorkError` renders only the message centred with the error colour — **no retry affordance** and no "something went wrong" recovery path. Trip-details error likewise shows plain text. See `06-QUALITY/ERROR-STATES.md`.

9. Offline behaviour (VERIFIED)

The app is **intrinsically offline**: the only operations are local Hive reads/writes and file picker access. Main `AndroidManifest.xml` declares **no** **`<uses-permission>`**, including no `INTERNET`. There is no network call path to fail. Consequences: - No permission requests are needed for core function (file export uses SAF scoped storage). - There is no sync, no retry-of-network, no cache-of-remote concept. "Offline mode" == normal mode.

10. Data ownership (current vs proposed)

- **Current (VERIFIED):** All records are owned by the single local app user; no ownership metadata, no audit log, no multi-user separation.
- **Native (PROPOSED):** Introduce server-authoritative owner/admin; per-record `ownerId`, `createdBy`, timestamps; audit logs in Firestore; role enforcement server-side.

11. Audit requirements (current vs proposed)

- **Current:** none — there is no audit trail of who did what; the app has no login.
- **Proposed (native):** `changedAt`, `changedBy`, immutable create records, admin audit log collection, and server-side functions for privileged operations.

12. Synchronisation requirements (current vs proposed)

- **Current:** none (offline). Backup file is the only export.
- **Native (PROPOSED):** evaluate Firestore offline persistence + online merge with conflict policy; do not silently overwrite.

13. Notification requirements (current vs proposed)

- **Current:** none. No notification code, no FCM, no local notifications.
- **Native (PROPOSED):** evaluate FCM + Cloud Functions + Firestore for role-scoped announcements; document permission strategy in `08-NATIVE-ANDROID/ANDROID-PERMISSIONS.md`.

14. Validation rules register

Field	Rule	Source
workType (add/edit form)	required	validator `v==null||v.isEmpty`
driver	required	validator
labours	≥1 required to save	`_saveWork` guard
tractor	required in `ConfirmNextTrip`	validator
driver (ConfirmNextTrip)	required	validator
Backup extension	must end `.labourbackup`	`_parseAndValidateBackup`
Backup size	≤ 25 MB	size check
Backup row counts	works≤10000, trips≤100000, labours≤5000	parse
Work date/session	editing requires matching work to preserve isolation	initState throw

15. Open operational questions

- OQ-4 Exact intended session boundary (code 04:00 vs PRD text 00:00). `UNVERIFIED`
- OQ-5 Whether trip deletion should be soft (history) or hard in future native. `UNVERIFIED`
- OQ-6 Required retention and audit depth for native. `UNVERIFIED`

01-PRODUCT/BUSINESS-RULES.md

Business Rules Register

Purpose: Every significant business/data rule with evidence and classification. Status legend per `00-AUDIT-INDEX.md`.

Status: VERIFIED unless labelled. Commit `2dd2fe4`.

Rules

BR-ID	Rule	Evidence	Status
BR-01	A `Work` is uniquely scoped by `(date, session)`; dashboard queries the current date + session	`dashboard_screen.dart`; `getWorkByDateAndSession`	VERIFIED
BR-02	A `Trip` belongs to exactly one `Work` (`workId`)	`trip.dart`, models	VERIFIED
BR-03	A `TripLabour` belongs to exactly one `Trip` (`tripId`) and references one `Labour` (`labourId`)	`trip_labour.dart`	VERIFIED
BR-04	Trip number: first=1; next = max(morning,evening)+1; edit preserves; delete does not renumber	`CalculateNextTripNumberUseCase`; tests	VERIFIED
BR-05	New date or session resets numbering baseline (computed per date)	usecase(date)	VERIFIED
BR-06	Labour `name` persists exactly as typed; must never auto-become `Labour N`	README/PRD; `_saveLabour`	VERIFIED
BR-07	Deleting a `Trip` cascade-deletes its `TripLabour` rows (composite-key prefix)	`work_local_data_source.dart deleteTrip`	VERIFIED
BR-08	Deleting/removing a labour from a trip must not delete the master `Labour` record	`DeleteTripLabourEvent` deletes join only; master separate	VERIFIED
BR-09	Attendance (`isPresent`) stored per trip per labour	model	VERIFIED
BR-10	Work id deterministic per `(date, session)`; saving repeatedly under same key aggregates trips to that Work	`_saveWork` `work_<date>_<session>`	VERIFIED
BR-11	Editing an existing trip must preserve `id`, `tripNumber`, `createdAt`, `date`, `session`	`SaveFullWorkTripEvent` `_onSaveFullWorkTrip` preserves; tests `date_partition_test`	VERIFIED
BR-12	Soft-delete semantics: labours removed from a full-trip save are marked `isPresent=false` (kept), not deleted	`_onSaveFullWorkTrip`	VERIFIED
BR-13	Backup file validation: `.labourbackup`, valid JSON, `app=="Labour Party"`, version+data present, ≤25 MB, row bounds	`settings_screen.dart`	VERIFIED
BR-14	Restore is all-or-nothing: snapshot → clear → apply → verify counts; rollback on failure	`settings_screen.dart`	VERIFIED
BR-15	Session boundary in code = 04:00–11:59 Morning, else Evening	`date_time_utils.dart`	VERIFIED
BR-16	(Doc claim) Session boundary "Morning 00:00–11:59"	Root `PRD.md` §36	CONFLICT with BR-15 → resolve
BR-17	Single user owns all records; no ownership metadata	codebase	VERIFIED
BR-18		model default; screens read-only	VERIFIED

BR-ID	Rule	Evidence	Status
	Trip `status` field exists, default `Completed`; no status workflow UI beyond read		

Native rules — PROPOSED

BR-ID	Rule
PBR-01	Ownership: every record has `ownerId`; labourer reads only own attendance
PBR-02	Role changes and privileged actions are server-enforced and audited
PBR-03	Client role flags are display hints only, never authorization
PBR-04	Trip/job status transitions restricted by role & current state
PBR-05	Offline writes queue and reconcile without silent overwrites

Verification status

VERIFIED BR-01..BR-18 except BR-16 (open conflict). PROPOSED PBR-01..05.

02-UX-UI/UX-AUDIT.md

UX Audit (roll-up)

Status: VERIFIED (static). Commit `2dd2fe4`. Detailed screen notes in `SCREENS.md`, flows in `UX-FLOWS.md`, gestures in `GESTURES.md`.

Overall UX posture

Cohesive, attractive dark glass dashboard app with clear hierarchy and fast data-entry affordances. Single-user/offline reduces many typical UX concerns (no auth/session/permission flows). Primary UX weaknesses are inactive/dead controls, an orphaned route, a misleading swipe affordance, dark-only theming, search limited to today, and accessibility gaps.

Strengths (VERIFIED)

- Fast "copy last trip" quick-add reduces repetitive entry.
- Summary → counter → list hierarchy on dashboard is clear.
- Consistent destructive-action confirmation; labour-remove has Undo.
- Draft autosave protects against data loss on field entry.
- Bottom-nav 4 tabs match the 4–5 guideline; no drawer clutter.

Findings roll-up

ID	Sev	Finding	Ref
UX-1	MED	`/details` route orphaned; search & filter inert; swipe delete affordance misleading	SCREENS S-03, BUG-02/03
UX-2	MED	Filter reachable only in state layer, not UI	BUG-04
UX-3	MED	Error states lack retry	ERROR-STATES
UX-4	MED	Search confined to today's session only	NAVIGATION-MAP
UX-5	LOW	Settings Theme/About tiles inert; empty-state blank CTA	BUG-06/07
UX-6	MED	Accessibility: icon buttons untagged, contrast risk	ACCESSIBILITY-AUDIT
UX-7	LOW	Dark-only; no light/dynamic theme	DESIGN-SYSTEM-AUDIT DS-4
UX-8	MED	Two overlapping "add trip" concepts (Add Work vs Next Trip)	PRD2 §3

Opportunities (PROPOSED)

Day-level "work detail", history-wide search+filter, per-role IA, bottom-sheet filters, consistent swipe-with-undo, light/dynamic theme, accessibility pass. See `08-NATIVE-ANDROID/NATIVE-UX-REDESIGN.md`.

Verification status

VERIFIED static. Motion/feel numeric validation UNVERIFIED (no device).

02-UX-UI/SCREENS.md

Screen Documentation (every screen)

Each screen includes the audited fields. Status: VERIFIED unless noted. Route map + back/drawer/bottom-nav analysis in `NAVIGATION-MAP.md`. Interaction details in `INTERACTION-SPECIFICATION.md`. Commit `2dd2fe4`.

> Accessibility, analytics and performance notes below reflect **current code**; proposed native treatments are in `08-NATIVE-ANDROID/`.

S-01 SplashScreen — route `/splash`

- **Purpose:** branded launch; performs async init (Hive + DI already done in `main()`); then navigates to dashboard.
- **Role / auth / authz:** single app user; none.
- **Entry:** app cold start (initial route). **Exit:** `/dashboard` after 1500 ms timer (`Future.delayed` → `context.go('/dashboard')`).
- **Parent/child:** none (root). Not in bottom-nav shell.
- **App bar:** none. **Back button:** none (system back exits).
- **Content hierarchy:** centred column: brand image `assets/branding/app_icon_192.png` (120px), app name "Labour Party", accent `CircularProgressIndicator`.
- **Components:** `Image.asset`, texts, progress; animations `fade/scale/slideY` via `flutter_animate`.
- **States:** no data states (single static screen). On nav timer completion it always goes to dashboard.
- **Gestures/animations:** entrance animations only.
- **Accessibility:** image has no semantic label (`Image.asset` alt text not set) — TalkBack reads filename/path. **Finding.**
- **Analytics:** none.
- **Security/perf:** trivial.
- **Status:** VERIFIED WORKING.

S-02 DashboardScreen — route `/dashboard` (bottom-nav tab 0)

- **Purpose:** home; today's Work + trips summary, quick-add, search, delete, list navigation.
- **Auth/authz:** none.
- **Entry:** `/splash` auto-nav; bottom nav. **Exit:** `/add-edit-work`, `/confirm-next-trip`, `/trip-details`, other tabs.
- **App bar:** custom: when not searching, brand image + "Labour Party" title; when searching, an inline `TextField` replaces the title. Actions: search/close toggle.
- **Back button:** none (top-level tab).
- **Bottom nav:** Dashboard/History/Analytics/Settings (via `MainLayout`).
- **Body states:** `WorkLoading/Initial → _buildSkeleton`; `WorkEmpty → EmptyStateWidget`; `WorkError →` centred message (no retry); `NavigateToConfirmNextTripState/WorkActionSuccess/TripDetailsLoaded → SizedBox` (routing/transient); `DashboardLoaded → _buildDashboard`.
- **Dashboard content hierarchy:** summary GlassCard (date • session, total trips chip, Morning/Evening stat items) → Current-Trip counter card (minus, number, plus) → "Recent Trips" header → trips list.

- **Recent trip list item:** `Dismissible` (swipe left to delete) wrapping `GestureDetector` → `GlassCard` row: trip-number avatar `#N`, tractor, "Driver: …", time.
- **Primary actions:** FAB "Add Work" (`/add-edit-work`, isNew) (shimmer animated), Current-trip plus (next trip), search.
- **Destructive actions:** minus (remove latest, disabled when no trips), swipe-to-delete with confirm dialog.
- **Search:** inline; on every change dispatches `SearchDashboardEvent`; toggling off clears + resets.
- **Pull-to-refresh:** `RefreshIndicator` reloads `LoadDashboardDataEvent`.
- **Loading/empty/error/offline:** offline = normal. Error no retry (finding).
- **Success:** after save, BLoC `WorkActionSuccess` triggers dashboard reload (this screen returns from push).
- **Gestures:** tap row → trip details; swipe left → delete; pull-to-refresh; FAB extended.
- **Empty CTA:** "Add First Trip" → `/add-edit-work`.
- **Accessibility:** Dismissible/Row semantics basic; no custom labels. **Finding** (contrast/semantics).
- **Analytics/security/perf:** none; in-memory + Hive reads; full list uses `ListView.builder` (shrinkWrap+non-scroll physics inside a scroll view).
- **Status:** VERIFIED WORKING.

S-03 DetailsScreen — route `/details` (in shell but not nav-visible)

- **Purpose:** shows current day's trip breakdown ("Work Details"). Appears to be a near-duplicate/legacy sibling of Dashboard's trip list.
- **Reachability:** route defined in `app_router.dart` inside the `ShellRoute`; **no visible UI navigates here** (`grep '/details'` only in router). Only deep-link/manual.
- **App bar:** title "Work Details"; actions: search icon toggle and a filter icon; **search `onChanged: (value) {}` does nothing and filter `onPressed: () {}` does nothing** → inert controls (VERIFIED PARTIAL/BROKEN).
- **Body:** uses `WorkBloc` with same states as dashboard; header stats (Date/Session/Trips) GlassCard + "Trip Breakdown" list.
- **List item:** `Dismissible` horizontal; background(s) are **blue edit icons** but `confirmDismiss` maps `endToStart→delete confirm`, else→edit → **misleading swipe affordance** (finding). Tap → `/trip-details`.
- **Edit:** `_editTrip` → `/add-edit-work` with isNew:false, editingTrip/work.
- **Auth/roles:** none.
- **Status:** PARTIAL — orphaned route + inert search/filter + misleading delete swipe. Recommend either wire it properly into nav or remove in native (`DEPRECATED`).

S-04 HistoryScreen — route `/history` (bottom-nav tab 1)

- **Purpose:** browse all recorded work grouped by date (desc) → session → trips.
- **Entry:** bottom nav. Loads via `LoadHistoryEvent` on `initState`. Refresh action reloads.
- **App bar:** "Work History"; actions: refresh icon.
- **Body states:** `HistoryLoading/Initial →` spinner; `HistoryError →` message; `HistoryEmpty → EmptyStateWidget` (note: empty CTA label, finding); `HistoryLoaded →` grouped list.
- **Grouped structure:** per-date `GlassCard` with `ExpansionTile` (first date initially expanded); inside, per-session header; per-trip `ListTile` (title `Trip #N - driver`, subtitle tractor, trailing edit + delete icons). Tap → `/trip-details`; edit → `/add-edit-work` (editingTrip/work); delete → confirm dialog then deletes + reloads history after 300 ms.
- **Auth/roles:** none.
- **Empty:** EmptyStateWidget; **CTA blank label** (shared component always draws button).
- **Status:** VERIFIED WORKING (with minor CTA-label issue).

S-05 AnalyticsScreen — route `/analytics` (bottom-nav tab 2)

- **Purpose:** compute KPIs & show sortable trip data table from history data.

- **Entry:** bottom nav → `LoadHistoryEvent`.
- **App bar:** "Analytics". Body states mirror History.
- **Content:** KPI row (Works / Trips / Top Driver) via three GlassCards; "Data Table" GlassCard with horizontally scrollable `DataTable` (Date, Session, Trip, Driver, Tractor). Column header tap sorts (date/session/trip/driver; ascending toggle; tractor not sortable).
- **Aggregation:** counts works/trips; driver & tractor ranking; builds flat list of trips (no labour loaded — memory note).
- **Empty:** EmptyStateWidget with blank CTA label (finding).
- **Auth/roles:** none.
- **Status:** VERIFIED WORKING (minor empty-state label issue).

S-06 SettingsScreen — route `/settings` (bottom-nav tab 3)

- **Purpose:** local backup/restore and (inert) theme/about.
- **Entry:** bottom nav.
- **App bar:** "Settings".
- **Body (loading):** full-screen spinner while backup/restore runs (`_isLoading`).
- **Sections:** GlassCard: "Backup Database" tile → `_backupDatabase`; "Restore Database" tile → `_restoreDatabase`. GlassCard: "Theme Mode" (shows "Dark", `onTap: () {}` inert); "About App" (`onTap: () {}` inert). Footer text: "Labour Party v1.0.0 Fully Offline".
- **Backup flow:** SAF save-file; JSON of all 4 boxes; success snackbar.
- **Restore flow:** SAF pick; extension check; off-thread parse/validate (25 MB, structure); summary+overwrite dialog; snapshot→clear→apply→verify counts→snackbar; rollback on failure; specific error dialogs (Too Large / Unsupported / Invalid / generic).
- **Auth/roles:** none.
- **Destructive:** restore overwrites current data (requires confirm dialog).
- **Accessibility:** file flows depend on platform pickers.
- **Status:** VERIFIED WORKING for backup/restore; Theme/About tiles PARTIAL/inert.

S-07 TripDetailsScreen — route `/trip-details` (pushed, extra: `Trip`)

- **Purpose:** per-trip detail: info, work info, timeline, labour/attendance management.
- **Entry:** tap trip row on dashboard/details/history. Loads `LoadTripDetailsEvent(trip.id)` on init.
- **App bar:** `Trip #N`; actions: edit (pencil) → `/add-edit-work` (isNew:false + editingTrip/work/labours) **only when state is `TripDetailsLoaded`** (otherwise silently does nothing — finding); add-labour (person_add) → dialog.
- **Body states:** spinner; then `TripDetailsLoaded` shows:
- Trip Info card (Tractor, Driver, Time, Status)
- Work Info card (Work Date, Session, Place, Work Type, Notes)
- Timeline card (Created, Last Modified, Duration mins)
- Labour Details: header + "Total/Present"; per-labour row (left accent border green=present/red=absent, name, edit icon, presence Switch, remove icon w/ Undo snackbar). If none: "No labours assigned to this trip".
- Add/edit labours via dialogs.
- **Presence toggle:** `UpdateTripLabourEvent`. **Remove:** `DeleteTripLabourEvent` + Undo → re-save.
- **Auth/roles:** none.
- **Destructive:** labour remove (with undo); no trip-delete from here.
- **Perf:** loads all labours list each time (`getLabours`) to resolve names (potential O(n×m) name lookups — see `05-PERFORMANCE/`).
- **Status:** VERIFIED WORKING (some dead/duplicate switch cases; name-edit refresh gap).

S-08 AddEditWorkScreen / "Add Trip" — route `/add-edit-work` (pushed, extra map)

- **Purpose:** create a new Work+Trip, or edit an existing trip/work.

- **Entry:** dashboard FAB/empty CTA (isNew true); details edit; trip-details edit; history edit (isNew false + editingTrip/work/labours).
- **App bar:** "Add Trip"; custom **back arrow** `leading` that saves draft then `context.pop()`.
- **Guard (VERIFIED):** if `editingTrip != null && editingWork == null` → throws Exception at init (defensive).
- **Body:** `Form` in `SingleChildScrollView`. Sections:
- Header GlassCard (Date, Session).
- Work Details: Work Type (required), Place (optional).
- Trip Details: tractor selector chips (Sonalika / JohnDeere) writing controller; Driver Name (required).
- Labour List: add-labour icon (dialog) + rows (name, presence Switch, remove). Empty → message "No labours added. Please add at least one."
- Save (PremiumButton "Save Trip"). Save guard: form valid AND ≥1 labour.
- **Draft autosave:** every controller change (and on lifecycle pause/inactive) writes a `DraftModel` into `draft_box` (only when creating, not editing); on open, if a draft exists shows "unsaved draft" snackbar with Restore.
- **Save action:** `_clearDraft()`; builds deterministic Work id; tripNumber 0 (auto) or preserved on edit; dispatches `SaveFullWorkTripEvent`; on `WorkActionSuccess` snackbar "Trip Saved Successfully!" and pops.
- **Auth/roles:** none.
- **Error:** invalid form inline; no-labour snackbar.
- **Status:** VERIFIED WORKING.

S-09 ConfirmNextTripScreen — route `/confirm-next-trip` (pushed, extra map incl. Work/nextTripNumber/previousLabours/place/workType)

- **Purpose:** review & save the "next trip" (copy of previous trip context + attendance).
- **Entry:** dashboard current-trip plus → BLoC `NavigateToConfirmNextTripState` → dashboard listener builds extras → push.
- **App bar:** "Confirm Next Trip". (System back = default pop; no unsaved-draft handling here.)
- **Body:** Form: Next Trip Overview card (work date/session, next trip #); Trip Details (Tractor required, Driver required, Work Type dropdown Sand (Bali)/Soil/Stone/Custom, Place, Notes); Labour & Attendance card (SwitchListTile per labour Present/Absent); Save ("Save as Next Trip") + Cancel.
- **Save:** builds new Trip (tripNumber = passed next) + TripLabour rows; dispatches `SaveNextTripEvent`; pops; snackbar; reload dashboard via listener on `WorkActionSuccess`.
- **Auth/roles:** none.
- **Status:** VERIFIED WORKING.

Cross-cutting notes

- **App bars:** all screens except splash have an AppBar; transparent bg, centred title (theme).
- **Back arrow:** default on pushed routes (S-07/08/09); custom on S-08. Top-level tabs (S-02/04/05/06) have no back.
- **Drawer/hamburger:** none anywhere (VERIFIED).
- **Bottom nav:** Dashboard/History/Analytics/Settings only (VERIFIED) — matches 4-destination guideline.
- **Inert controls finding:** Settings Theme/About; Details search/filter; AddQuickTripEvent handler empty.

Navigation Map & Route Register

Purpose: complete navigation graph, route register, and back/drawer/bottom-nav analysis. Status: VERIFIED. Commit `2dd2fe4`.

1. Router (VERIFIED)

`go_router` config in `lib/routes/app_router.dart`. Initial route: `/splash`. Root navigator + one `ShellRoute` (nested navigator) hosting the bottom-nav tabs.

Route register

Route	Screen	In shell	Navigation	Extra payload
`/splash`	S-01	no	initial	—
`/dashboard`	S-02	yes (tab0)	`go` from splash/nav	—
`/details`	S-03	yes (no tab)	unreachable from nav	—
`/settings`	S-06	yes (tab3)	nav	—
`/history`	S-04	yes (tab1)	nav	—
`/analytics`	S-05	yes (tab2)	nav	—
`/trip-details`	S-07	no	`push`	`Trip`
`/add-edit-work`	S-08	no	`push`	Map (`isNew` , `editingTrip` , `editingWork` , `editingLabours`)
`/confirm-next-trip`	S-09	no	`push`	Map (`work` , `nextTripNumber` , `previousLabours` , `place` , `workType`)

Navigation method (VERIFIED)

- Tabs & post-splash use `context.go(...)` (replace, resets back stack to the tab).
- Sub-pages use `context.push(...)` (stack). Returns via `context.pop()`.
- State-carrying navigation uses `state.extra`; **TripDetailsScreen reads `state.extra as Trip`** and casts directly → deep-linking without extra would crash (`UNVERIFIED` robustness). Note: GoRouter by default passes `extra` only for in-session pushes; URL-based navigation of `extra`-bearing routes is fragile.

```
flowchart TD
  ROOT[/splash/] -->|go| DASH
  subgraph SHELL[ShellRoute / MainLayout bottom-nav]
    DASH[/dashboard/]
    DET[/details - orphaned/]
    HIST[/history/]
```

```
    ANAL[/analytics/]
    SET[/settings/]
  end
  DASH -->|push| TRIP
  DASH -->|push| ADD
  DASH -->|push| CONFIRM
  DET -->|push| TRIP
  DET -->|push| ADD
  HIST -->|push| TRIP
  HIST -->|push| ADD
  TRIP -->|push| ADD
```

2. Bottom navigation (VERIFIED)

`MainLayout` uses Material 3 `NavigationBar` with 4 destinations: Dashboard(0), History(1), Analytics(2), Settings(3). `selectedIndex` derived from current path; tapping calls `context.go`. No drawer. This matches the 4–5 destination guidance.

3. Back arrow / system back behaviour (VERIFIED)

- Pushed screens (Trip Details, Add/Edit, Confirm Next Trip): leading back arrow defaults or is custom; system back pops.
- S-08 custom back saves the draft before `pop()` (good).
- S-09 pops immediately after dispatching save (relies on BLoC success handling separately).
- Top-level tabs: no leading back (system back exits app from shell).
- No `PopScope` handling for unsaved changes on S-09 (back discards silently); S-08 has draft autosave instead.
- **Unsaved-changes behaviour:** Add/Edit autosaves draft; Confirm Next Trip has no draft & system back discards. Noted for native `PopScope`.

4. Deep links (VERIFIED/MISSING)

No deep-link scheme/App Links declared; no intent-filter routes beyond launcher; `extra`-based navigation isn't deep-link-safe. **Native:** evaluate App Links only if a companion web/product needs them.

5. Drawer / hamburger (VERIFIED — none)

No `Drawer`, no hamburger icon anywhere. Navigation is bottom-nav only.

6. Recommended native navigation model (PROPOSED)

- Jetpack Navigation Compose with 4–5 top-level destinations via bottom bar (roles may change the set).
- Owner home: Dashboard, Crew/Work, Attendance, Reports, Settings (+ notifications via bell).
- Driver home: My Jobs, Status, Profile.
- Labourer home (if separate): My Attendance.
- Use `SavedStateHandle`/ViewModel to survive process recreation; deep-link safe IDs (never rely on passing whole objects via extras).

7. Cross-doc consistency

Routes above match `SCREENS.md` S-IDs. Orphan `/details` flagged.

02-UX-UI/GESTURES.md

Gestures Audit

Purpose: document every implemented gesture and propose native gestures (PROPOSED), each with a usability justification. Status: VERIFIED unless PROPOSED. Commit `2dd2fe4`.

Implemented gestures (VERIFIED)

Gesture	Location	Trigger/behaviour	Justification
Tap	Dashboard trip row	push `/trip-details`	standard drill-in
Tap	History trip row / icons	details / edit / delete	standard
Tap	Analytics column header	sort asc/desc	data table sorting
Tap	Dashboard current-trip **+**	add next trip	quick add
Tap	Dashboard current-trip **−**	remove latest (confirm)	undo-last
Long-press	none	—	not used
Swipe left (end-to-start)	Dashboard trip list (`Dismissible`)	delete (confirm)	quick delete
Swipe horizontal	Details trip list	`endToStart` →delete, else→edit	edit/delete (but see finding: misleading affordance, both backgrounds show edit icon while delete triggers red dialog)
Pull-to-refresh	Dashboard	reload current session	freshness
Scrolling	All lists/forms	standard	content paging
Drag	none	—	not used
Pinch/zoom	none	—	N/A
Dismiss/bottom-sheet/drawer gestures	none	no sheets/drawers	N/A
Keyboard	Search, forms	typing, Enter	standard
Back edge/system	Android	pop / exit	platform

Findings (VERIFIED)

- G-1: Details screen `Dismissible` reveals a **blue edit icon** for a gesture that maps to **delete**; misleading. Recommended fix in native: background colour/icon must match the mapped action (delete=red trash, edit=blue/pencil) and swipe direction semantics consistent with dashboard.
- G-2: No undo for trip delete (only labour remove has Undo). Consider Undo for trip delete (PROPOSED).
- G-3: Dashboard list is a `ListView.builder` inside a scroll view with `NeverScrollableScrollPhysics` — vertical scrolling of the whole page, not the inner list; acceptable but note nested-scroll behaviour.

Proposed native gestures (PROPOSED)

Gesture	Where	Justification
Tap with proper ripple + haptics	buttons/rows	tactile feedback
Pull-to-refresh on lists backed by server	owner/driver job lists	data freshness
Swipe actions w/ role-gated delete/archive & Undo	job/attendance rows	fast triage w/ recovery
Bottom-sheet for filter/role menus	dashboard filters	contextual, avoids nav clutter
Standard LazyColumn scroll + scroll-to-top	long lists	perf & UX
Back/swipe gesture	Android predictive back	platform-native

Rule

Only add gestures with clear usability + role-permission justification. Do not add cosmetic-only gestures.

ENGINEERING ARCHITECTURE

03-ENGINEERING/ARCHITECTURE.md

Architecture (current) — overview

Status: VERIFIED. Commit `2dd2fe4`.

Current stack

- **Language/SDK:** Dart 3 (sealed classes), Flutter; env `sdk: ^3.11.0`.
- **UI:** Flutter widgets + Material 3 dark theme + ScreenUtil + go_router.
- **State:** flutter_bloc (BLoC pattern) with sealed state classes (Cubit-like event handlers).
- **DI:** get_it (`lib/config/di/injection_container.dart`).
- **Domain/error handling:** dartz `Either<Failure,T>` + typed `Failure` (`core/error/failures.dart`).
- **Persistence:** Hive NoSQL (5 boxes).
- **ID generation:** uuid.
- **Clean-Architecture-inspired layering:** presentation / (bloc) / domain / data.

Layering & dependency direction

```
flowchart TD
  UI[presentation: screens + widgets] --> BLOC[bloc: events/state]
  BLOC --> UC[domain: usecases]
  UC --> REPO[domain: repository interface]
  REPO --> IMPL[data: repository impl]
  IMPL --> DS[data: local data source]
  DS --> HIVE[Hive boxes]
  DI[get_it] -- wires --> BLOC & REPO & DS & HIVE
```

Direction: presentation → domain ← data (domain is the center). UI does not touch Hive except a few pragmatic exceptions (draft save/restore and dashboard next-trip labour-name lookup read boxes directly) — **layering exceptions noted** (see `CURRENT-ARCHITECTURE.md`).

Boundaries honoured vs violated (VERIFIED)

Honoured: - Repository abstraction isolates Hive; usecases hold business rules (trip numbering). - Bloc mediates UI ↔ usecases; sealed states give exhaustive switches.

Violated / pragmatic: - `dashboard_screen.dart` reads `Hive.box<LabourModel>(labourBox)` directly in the next-trip listener to resolve names (presentation → data). - `add_edit_work_screen.dart` reads/writes `draft_box` directly (Hive) for draft autosave. - Analytics/history compute some aggregation inside the screen (presentation logic). - Duplicate `LabourFormModel` is defined in two screen files (shared-form-model duplication, not a shared layer).

Strengths & weaknesses

- Strengths: clear separation for domain data rules; typed failures; deterministic testing hooks (mock repo).

- Weaknesses: presentation-layer pragmatism leaks; no use of a formal state container beyond Bloc events; several dead/inert paths; no persistence versioning/migration framework beyond bespoke legacy-key migration; offline single-user.

Native target architecture

See `08-NATIVE-ANDROID/ANDROID-ARCHITECTURE.md` (Kotlin + Jetpack Compose + M3 + Clean Architecture + Hilt + Coroutines/Flow + UDF/MVI). The current repo is the **behavioural reference**, not the architectural blueprint.

03-ENGINEERING/CURRENT-ARCHITECTURE.md

Current Architecture — detailed

Status: VERIFIED. Commit `2dd2fe4`.

Directory structure

```
lib/
├── main.dart                     # boot: Hive init -> DI init -> runApp
├── routes/app_router.dart        # go_router config (splash/shell/sub-routes)
├── theme/app_theme.dart          # M3 dark theme + tokens
├── config/di/injection_container.dart  # get_it wiring
├── core/
│   ├── database/hive_setup.dart # opens 5 boxes + registers adapters
│   ├── error/failures.dart      # Failure, DatabaseFailure, ValidationFailure
│   ├── usecases/usecase.dart    # UseCase<T,P> (dartz Either)
│   └── utils/date_time_utils.dart
├── features/
│   ├── dashboard/presentation/{splash,dashboard}_screen.dart
│   ├── history/presentation/history_screen.dart
│   ├── analytics/presentation/analytics_screen.dart
│   ├── settings/presentation/settings_screen.dart
│   ├── details/presentation/{details,trip_details}_screen.dart
│   └── work/
│       ├── data/{datasources,models,repositories}/
│       ├── domain/{entities,repositories,usecases}/
│       └── presentation/{bloc,screens}/
├── shared/
│   ├── main_layout.dart
│   └── widgets/ (empty_state, glass_card, custom_text_field, premium_button, skeleton_loading)
```

Startup sequence

`main()` → `WidgetsFlutterBinding.ensureInitialized()` → `HiveSetup.init()` (registers 5 adapters, opens boxes) → `di.init()` → `runApp(MyApp)` → `ScreenUtilInit` → `MultiBlocProvider(WorkBloc, HistoryBloc)` → `MaterialApp.router`.

Feature wiring

- `WorkBloc` is constructed with 15 usecases + a uuid; owns search query & filter private fields.
- `HistoryBloc` uses `GetWorksUseCase` + `GetTripsForWorkUseCase`.

- Repository impl maps `*Model → *Entity`; Data source owns Hive box keys & migration (`_migrateLegacyTripLabours`).
- Backups read Hive boxes directly in Settings (data-layer code embedded in a presentation screen).

Concurrency & lifecycle

- Hive boxes opened once at startup and held as singletons.
- Box operations are async `put/get/delete`; no explicit transactions (restore emulates atomicity with snapshot+rollback).
- No background workers, no WorkManager (not needed offline single-user).

Known architectural smells

ID	Smell	Evidence
A-1	Presentation accesses Hive directly	`dashboard_screen.dart`, `add_edit_work_screen.dart`
A-2	Business aggregation in presentation	`analytics_screen.dart`, `history_screen.dart` grouping
A-3	Duplicate `LabourFormModel` type across two screens	both create screens
A-4	Duplicate `@override` annotations & duplicate switch cases	`work_local_data_source.dart`, `work_repository_impl.dart`, `trip_details_screen.dart`
A-5	Backup/restore logic (data) inside a screen	`settings_screen.dart` top-level funcs
A-6	Empty event handler `AddQuickTripEvent`	`work_bloc.dart`

Verification status

VERIFIED against code. Native re-architecture is separate (PROPOSED).

03-ENGINEERING/DATABASE.md

Database (current — Hive)

Status: VERIFIED. Commit `2dd2fe4`.

Engine

Hive (`hive` + `hive_flutter`). NoSQL key-value document store; boxes opened in `lib/core/database/hive_setup.dart`. Files live in the app-support directory resolved by `path_provider` via `Hive.initFlutter()`.

Boxes

Box constant	Generic type	Adapter typeId	Purpose	Primary key scheme
`work_box`	`WorkModel`	0	top-level works	work id (`work_<date_>_<session>` for created; uuid otherwise)
`trip_box`	`TripModel`	1	trips	trip id (uuid)

Box constant	Generic type	Adapter typeId	Purpose	Primary key scheme
`labour_box`	`LabourModel`	2	labourer master records	labour id (uuid)
`trip_labour_box`	`TripLabourModel`	3	per-trip attendance joins	composite `"<tripId>_<tripLabourId>"`
`draft_box`	`DraftModel`	4	unsaved new-trip form	fixed key `'current_draft'`

TypeId registry: WorkModel=0, TripModel=1, LabourModel=2, TripLabourModel=3, DraftModel=4 (Hive object versioning uses typeId + field index; fields must not be renumbered).

Schema (fields) — see `DATA-MODEL.md` for detail

Each model maps 1:1 to its entity. `TripModel` fields carry Hive default values for forward/backward compat (`place=''`, `workType='Sand (Bali)'`, `notes=''`, `updatedAt=null`, `status='Completed'`).

Keying / performance strategy (VERIFIED)

- `TripLabour` stored by **composite key** `"<tripId>_<id>"` to allow prefix scans: `getLaboursForTrip` filters keys starting `"<tripId>_"` (avoids full O(N) value scan); `deleteTrip` deletes the trip then prefix-deletes its TripLabour keys.
- Bulk writes use `putAll` (e.g., `saveTripLabours`) which deletes existing prefix keys first then inserts, to avoid orphans on edit.
- Legacy-key migration: on data-source construction, keys without `_` are copied to composite keys and removed.

Integrity operations

- Cascade delete trip → attendance.
- Restore implements snapshot → clear → apply → verify → rollback.

Findings (VERIFIED)

- No formal schema version/migration framework (only the bespoke legacy-key migration + Hive default values). Future model changes need careful field-index handling.
- No encryption at rest (Hive default). Labour names/phones stored in plaintext within app sandbox.
- Draft box stores JSON-encoded labour list in a string field (`encodedLabours`) — schema-in-string anti-pattern.
- Deterministic work-id by date+session means multiple "Add Work" saves on the same session reuse the same work key (aggregation) while still creating distinct trips — confirm intended semantics.

Native notes

For native, evaluate **Room** (relational) where joins/cascade/queries are needed, or Firestore (see `08-NATIVE-ANDROID/FIREBASE-DATABASE.md`). Map each Hive box to a schema; version & migrate via Room migrations or Firestore structure.

03-ENGINEERING/DATA-MODEL.md

Data Model (current)

Status: VERIFIED. Commit `2dd2fe4`. Entities in `lib/features/work/domain/entities/`, models in `.../data/models/`.

Entities & models

Work (`work.dart` / `WorkModel` typeId 0)

Field	Type	Required	Notes
id	String	yes	created: `work_<date spaces→_>_<session>`; else uuid
date	String	yes	`dd MMM yyyy`
session	String	yes	Morning/Evening
workType	String	yes	default seed `Sand (Bali)`
place	String	optional default ''	
createdAt	DateTime	yes	
updatedAt	DateTime	yes	

Trip (`trip.dart` / `TripModel` typeId 1)

Field	Type	Required/default	Notes
id	String	yes	uuid
workId	String	yes	FK → Work
tripNumber	int	yes	derived; edit preserves; 0=auto on create
tractor	String	yes	e.g., Sonalika/JohnDeere
driverName	String	yes	
createdAt	DateTime	yes	
place	String	default ''	
workType	String	default 'Sand (Bali)'	
notes	String	default ''	
updatedAt	DateTime?	default null	
status	String	default 'Completed'	no status workflow UI

Labour (`labour.dart` / `LabourModel` typeId 2)

Field	Type	Notes
id	String	uuid
name	String	persists exactly as typed

Field	Type	Notes
phoneOptional	String?	optional PII
createdAt	DateTime	

TripLabour (`trip_labour.dart` / `TripLabourModel` typeId 3)

Field	Type	Notes
id	String	uuid
tripId	String	FK → Trip
labourId	String	FK → Labour
isPresent	bool	attendance

Draft (`draft_model.dart` typeId 4)

Field	Type	Notes
date/session/workType/place/tractor/driverName	String	
encodedLabours	String	JSON string of `[{labourId,name,isPresent}]`

Relationships

```
Work 1 ── * Trip 1 ── * TripLabour * ── 1 Labour
Work * ── * Labour   (via TripLabour; Labour is a global master list)
```

Foreign keys are string references (no DB-level enforcement) — integrity enforced in repository/data source & usecase logic.

Data integrity rules implemented

- Trip belongs to one Work; TripLabour to one Trip + one Labour.
- Delete trip → delete its TripLabour (prefix).
- Full-trip save marks removed labours `isPresent=false` (soft) rather than deleting (per PRD).
- Restore validates row counts and rolls back.

PII inventory (VERIFIED)

- `Labour.name`, `Labour.phoneOptional` (optional phone) are personal data stored locally & exported unencrypted in backups. No other PII (no DOB, addresses, government IDs).
- Backup file also contains `createdAt` times and driver names.

Findings / gaps (VERIFIED)

- Labour `phoneOptional` exists in the model but **no UI captures phone** (search/absence confirmed) — field unused by current UI. Confirm intent.
- Trip `status` fixed to default; no status state machine in UI.
- No ownership/audit fields (single-user by design).
- Duplicate definition of the presentation form model `LabourFormModel` in two screen files (not part of data model).

- `TripLabour.updatedAt` absent (no attendance-change timestamp) — attendance audit history not preserved (only latest state).

Verification status

VERIFIED from models/entities. Future ownership/audit fields are PROPOSED.

03-ENGINEERING/STATE-MANAGEMENT.md

State Management (current)

Status: VERIFIED. Commit `2dd2fe4`.

Approach

flutter_bloc. Two blocs provided app-wide in `main.dart`: - `WorkBloc` — dashboard/trips/labours/attendance + navigation-for-next-trip orchestration. - `HistoryBloc` — history & analytics data.

Sealed state + event classes (`work_state.dart`, `work_event.dart`, `history_*.dart`). Emitters drive UI via `BlocConsumer` / `BlocBuilder` / `BlocListener`.

WorkBloc state set

`WorkInitial, WorkLoading, DashboardLoaded{currentWork,currentTrips,totalLabourCount,morningTripCount,eveningTripCount,totalTrips,searchQuer TripDetailsLoaded{work,tripLabours,labours}, WorkEmpty{message}, WorkError{message}, WorkActionSuccess, NavigateToConfirmNextTripState{...}`.

WorkBloc events

LoadDashboardData, AddQuickTrip (empty handler — dead), NavigateToConfirmNextTrip, SaveNextTrip, RemoveLatestTrip, DeleteSpecificTrip, LoadTripDetails, SaveFullWorkTrip, SaveTripLabour, SaveLabour, UpdateTripLabour, DeleteTripLabour, SearchDashboard, FilterDashboard.

Important behaviours / smell

- `_onSearchDashboard` sets `_currentSearchQuery` (a private field), then re-dispatches `LoadDashboardDataEvent`; filtering is applied inside the dashboard-load handler using the private field. **Search/filter state lives in BLoC private mutable fields, not in the emitted state** (`DashboardLoaded.searchQuery` is echoed but is excluded from `props`, so it doesn't drive rebuild equality).
- `FilterDashboardEvent` handler stores `_currentFilters` but the load path never actually applies filter predicates (`matchesFilter=true` always) → filtering effectively unimplemented (PARTIAL).
- Navigation-to-next-trip is emitted as a **state** (`NavigateToConfirmNextTripState`) that the dashboard listener reacts to by pushing a route — navigation-as-state coupling.
- After emitting `NavigateToConfirmNextTripState`, `_onNavigateToConfirmNextTrip` re-emits the captured `DashboardLoaded` — multi-emit per event.
- `WorkActionSuccess` is a transient state consumed by listeners to pop/snackbar.

HistoryBloc

`LoadHistoryEvent` → `HistoryLoading → HistoryLoaded{groupedTrips:Date→Session→Trips, worksMap}/HistoryEmpty/HistoryError`. Aggregation (grouping) is done in the bloc.

Loading/empty/error surfaced

- Screens switch exhaustively on sealed states. Several screens render transient states as `SizedBox.shrink` (routing states) — acceptable but note.

Findings

ID	Finding	Severity
S-1	Mutable private filter/search state not reflected in state equality	MEDIUM
S-2	Filter predicates not implemented	MEDIUM
S-3	Empty `AddQuickTripEvent` handler	LOW
S-4	Navigation triggered from state listener	MEDIUM (architecture)
S-5	Multi-emit per navigation event	LOW

Native

Recommended UDF/MVI: single `UiState` in `StateFlow` with explicit Loading/Success/Empty/Error/Offline/Unauthorized/Forbidden; one-way events; avoid navigation-as-state (see `08-NATIVE-ANDROID/ANDROID-ARCHITECTURE.md`).

03-ENGINEERING/ANDROID-PLATFORM.md

Android Platform (current Flutter host)

Status: VERIFIED (static). Commit `2dd2fe4`.

Host configuration

Item	Value	Source
Package/namespace/applicationId	`com.roshan.labourparty`	`build.gradle.kts`
Flutter/Dart SDK floor	`sdk: ^3.11.0`	`pubspec.yaml`
Flutter revision (`.metadata`)	`90673a4eef...` (channel stable)	`.metadata`
minSdk	24	`build.gradle.kts`
compileSdk	`flutter.compileSdkVersion` (Flutter default)	`build.gradle.kts`
targetSdk	`flutter.targetSdkVersion`	`build.gradle.kts`
Android Gradle Plugin	`8.11.1`	`settings.gradle.kts`

Item	Value	Source
Kotlin (Gradle)	`2.2.20`	`settings.gradle.kts`
Gradle	`8.14`	`gradle-wrapper.properties`
Java toolchain	17	`build.gradle.kts`
versionName / versionCode	`1.0.0` / `1` (from pubspec `1.0.0+1`)	pubspec
Release	minify+shrink true, proguard file present; signing from `key.properties` (not committed)	`build.gradle.kts`

Platform scaffolding present

- Only `android/` is fully scaffolded; no `ios/`, `web/`, `linux/`, `windows/`, `macos/` directories committed (`.metadata` lists them but directories absent).
- `MainActivity.kt` = default `FlutterActivity`.
- Launcher icons & launch/normal themes under `res/`.

Manifest (VERIFIED)

- **No `<uses-permission>` entries** — including no `INTERNET`. Confirms offline-only & no runtime permissions.
- `<queries>` for `PROCESS_TEXT` (Flutter default). Activity `android:exported="true"` with MAIN/LAUNCHER filter; `configChanges` covers orientation/keyboard/screenSize/locale etc.; `windowSoftInputMode=adjustResize`.
- No deep-link intent filters; no exported components beyond launcher.

Release signing (VERIFIED)

- Signing uses `android/key.properties` (git-ignored; only `key.properties.example` committed). Release task throws if key.properties missing.
- **CRITICAL finding:** `android/app/keystore.jks.bak` (2,690-byte DER file; begins `30 82 ...` = ASN.1; contains a PKCS#8 v0 `02 01 00` private-key marker; named "keystore...bak") is **tracked in the repository** (added in commit `55b2144`). Its name implies a signing/upload keystore backup. `.gitignore` ignores `*.jks` / `*.keystore` but **not** `*.jks.bak`, so it slipped through. Treat as potential private-key exposure → see `04-SECURITY/SECRETS-AUDIT.md`. (Only structural facts reported; contents not reproduced.)

Findings

ID	Finding	Severity
AND-1	Keystore-like private-key file committed (`.jks.bak`)	CRITICAL
AND-2	compileSdk/targetSdk not pinned (Flutter defaults)	LOW
AND-3	No versionCode build automation/config flavours	LOW
AND-4	No CI/CD pipeline config in repo	MEDIUM (see 07-OPERATIONS)

Native (PROPOSED)

Pin compileSdk/targetSdk/minSdk deliberately; define build variants & flavours if needed; use Play App Signing; store signing key outside repo; enable App Check when Firebase added. See `08-NATIVE-ANDROID/`.

SECURITY AUDIT

04-SECURITY/SECURITY.md

Security Overview (current + target)

Status: current VERIFIED; native controls PROPOSED. Commit `2dd2fe4`.

Executive position

The **current** product has a very small attack surface because it is **offline, single-user, local-only, unauthenticated**, with no network, no backend, and no secrets required at runtime. Its main risks are local/data-level and configuration-level (a committed keystore-like file). It is **not** a multi-user/RBAC product today.

The **native** product introduces accounts, roles, a backend, notifications, and cloud data — a fundamentally larger trust boundary that requires server-authoritative security. This must be designed in from the start, not bolted on.

Guiding principle (applies to native)

> **«Client-side role checks are NOT sufficient authorization.» The backend must enforce authorization.**

Current security posture (VERIFIED)

Area	Status	Notes
Authentication	NONE (by design)	single local user
Authorization	NONE	all features available to the app user
RBAC	NONE	no roles exist
Network exposure	NONE	no INTERNET permission; no sockets/http
Secrets in runtime code	NONE	client holds no keys
Secrets in repo	**YES (CRITICAL)**	`android/app/keystore.jks.bak` committed (see SECRETS-AUDIT)
Encryption at rest	NONE	Hive plaintext
Backup security	Weak	plaintext JSON `.labourbackup`; structural validation only
Input validation	Present (form validators, backup parse)	client-side
Deep links/exported components	Minimal	launcher only; no exported receivers
Logging	No production logging/analytics	offline

Target security architecture (PROPOSED)

- Firebase Authentication (owner/admin + optional driver/labourer).
- Backend-authoritative roles (custom claims or role documents/Cloud Functions) — never client `isAdmin=true`.
- Firestore + Storage security rules; App Check; Cloud Functions for privileged/transactional ops.
- Audit logging for admin/privileged actions.
- Encryption at rest for local PII where applicable; authenticated & rule-protected cloud backup.
- Enforce: IDOR protection, privilege escalation prevention, rate limiting, replay controls.

See `04-SECURITY/THREAT-MODEL.md`, `SECURITY-AUDIT.md`, `AUTHENTICATION-AUTHORIZATION.md`, `RBAC.md`, `SECRETS-AUDIT.md`, `PRIVACY-DATA-FLOW.md`.

04-SECURITY/SECURITY-AUDIT.md

Security Audit (current)

Status: VERIFIED (static). Severity: CRITICAL/HIGH/MEDIUM/LOW/INFO. Commit `2dd2fe4`.

Findings

ID	Sev	Finding	Evidence	Remediation
SEC-1	CRITICAL	Release-keystore-like backup (`keystore.jks.bak`, DER/ PKCS#8 v0 structure) committed to repo	tracked file; added commit `55b2144`; `.gitignore` misses `*.jks.bak`	Rotate/regenerate signing key; remove file from history; expand .gitignore; if it is a live key, treat as compromised & revoke Play key via Play App Signing reset
SEC-2	MEDIUM	Local data & backup unencrypted PII (labour names, optional phone)	Hive plaintext; plaintext `.labourbackup`	Encrypt local PII at rest (native); signed/encrypted backup
SEC-3	MEDIUM	Restore trusts file contents after structural checks (no authenticity/owner integrity); could inject arbitrary records	`_parseAndValidateBackup`	Add integrity signature / authenticator of backups
SEC-4	LOW	No output sanitization needed (no HTML/webview), but error messages concatenate raw exception text shown to user	repo impls `...$e`	Provide clean, non-internal error text
SEC-5	LOW	Duplicate `@override` / duplicate `switch` cases indicate risky manual merges	data source, repo impl, trip_details	Clean; add lint/test gate
SEC-6	INFO	No authentication by design; not a vuln today but becomes one if app is opened up to multi-user without adding authz	whole app	Native must add auth + server authz
SEC-7	INFO	No crash reporting/analytics (offline) — no telemetry risk, but also no visibility	none	Native adds Crashlytics w/ non-PII config

In scope checks with no issue found

- Injection: no SQL (Hive only); no code-eval paths; form strings used as data.
- WebViews: none present.
- Exported components: only launcher MainActivity.
- Deep links: none.
- Network/TLS: not applicable (no network).
- Token/session storage: not applicable (no auth).

Positive

- No runtime API keys/secret strings in `lib/`; `key.properties` correctly git-ignored; release task fails safely if key.properties missing.

Verification status

Static VERIFIED. Runtime/pen-testing not possible in this environment (`UNVERIFIED` dynamic).

04-SECURITY/THREAT-MODEL.md

Threat Model

Status: current + proposed. Commit `2dd2fe4`. STRIDE-based.

1. Current product (offline single-user)

Assets: work/trip/labour records (incl. names & optional phones) on device; backup files; app availability; device identity.

Threats

Threat	STRIDE	Likelihood	Impact	Mitigation
Device loss/theft exposes plaintext PII (Hive)	Information disclosure	MED	HIGH	OS sandbox; native: encrypt at rest
Backup file stolen/plaintext leaks names/phones	Information disclosure	MED	MED	Secure backup location; native: encrypted/signed
Malicious crafted `.labourbackup` injects arbitrary records on restore	Tampering	LOW	MED	Structural validation + count check (exists); native: integrity/authenticity
Repo keystore exposure (SEC-1) → signing-key theft → malicious signed builds	Spoofing/Repudiation	MED (already occurred)	HIGH	Rotate key; purge history; Play App Signing
Reverse-engineering of local DB	Information disclosure	MED	LOW-MED	Encryption at rest (native)
App process crash mid-restore corrupts DB	DoS/data-loss	LOW	MED	Snapshot+rollback (exists)
No updates if no distribution channel	Availability	LOW	LOW	Not in current threat control

Attacker profile: device thief, casual reverse-engineer, person with a crafted backup. No network attacker (no network).

2. Native product (multi-user + backend) — PROPOSED

New assets: user accounts/roles, cloud Firestore/Storage data, notifications, audit logs.

New threats & required controls

Threat	Control
Unauthorized user reads others' data (IDOR)	Firestore rules scoping reads by role/ownerId; never trust client ids blindly
Client claims `isAdmin=true`	Backend-authoritative role (custom claims/role doc); rules check auth.uid role
Privilege escalation (labourer→admin)	Server-side role assignment only via Cloud Functions/admin
Stolen/leaked FCM token or session	Token lifecycle mgmt; App Check; secure token storage (Keystore)
Firestore/Storage abuse (quota, over-read)	Rules + rate limits + Cloud Functions caps
Replay / race on trip/status updates	Transactions + idempotency keys server-side
Admin actions without trace	Audit log collection written by backend only
Notifications abuse	Cloud Functions to send; role-scoped topics; auth checks
Malicious client writes invalid domain data	Cloud Functions/security rules validate domain invariants
Backup/DR compromise	Authenticated, rule-protected Cloud Storage; retention; restore verification

3. Data-flow trust boundaries (native, PROPOSED)

Client → (App Check) → Firebase Auth → Firestore/Storage (rules) / Cloud Functions (authz+validation) → backend systems. Client never bypasses rules.

4. Open items

- OQ-7 Whether driver/labourer roles get their own login (scope/consent). UNVERIFIED
- OQ-8 Required granularity of audit logging & retention. UNVERIFIED

04-SECURITY/SECRETS-AUDIT.md

Secrets Audit

Purpose: locate and classify any secrets/keys in the repository **without reproducing their values**.

Status: VERIFIED. Commit `2dd2fe4` .

Policy (repeated from brief)

Absolutely prohibited: API secrets, passwords, service-account credentials in APK, private keys in repo, production tokens in config, Firebase Admin credentials in Android, hardcoded owner credentials.

Findings

ID	Sev	Location	Description	Action
SA-1	CRITICAL	`android/app/keystore.jks.bak`	2,690-byte **tracked** file. Hex prefix `30 82` = ASN.1 DER SEQUENCE; contains a PKCS#8 version-0 marker (`02 01 00`). Filename implies a keystore/signing-key **backup**. `.gitignore` excludes `*.jks` / `*.keystore` but NOT `*.jks.bak`. Added in commit `55b2144` and **present in every tag** (`v1.0.0`, `v1.0.0-rc1`, `v1.0.1-hotfix-rc1`, `RC-1.3`).	Do NOT treat as benign. (1) Determine whether this is an active signing/upload key. (2) Regenerate & rotate the key. (3) Purge the file and its history across all branches/tags (e.g., `git filter-repo` + tag rewrite). (4) Harden `.gitignore`. If it is the Play upload key, use Play App Signing to reset the key. Values are NOT reproduced in this audit.
SA-2	NONE	`android/key.properties`	Correctly **not committed** (absent). Only `key.properties.example` (blank template) is tracked.	None — good practice.
SA-3	NONE	source code (`lib/`)	No hardcoded credentials/owner password. No owner name present.	None.
SA-4	INFO	`android/app/proguard-rules.pro`	`-keep class com.roshan.labourparty.**`	Not a secret; confirm intended for R8.

Confirmed clean (VERIFIED)

- No Firebase/Admin JSON, no `.p12`, no `.env`, no service-account files, no API tokens found under `lib/`, `android/` (other than SA-1), `assets/`.
- No hardcoded owner credentials anywhere.
- `key.properties` git-ignored.

Remediation checklist (PROPOSED)

- [ ] Treat SA-1 as potential private-key exposure; rotate immediately if live.
- [ ] Remove SA-1 from repo & history.
- [ ] Add `*.jks.bak`, `*.keystore.bak`, `**/*.jks`, `**/*.keystore`, `key.properties*` to ignore; add a secret-scan to CI (gitleaks/TruffleHog).
- [ ] Audit other branches/tags for the file — **DONE (VERIFIED): present in all four tags.**

Verification status

VERIFIED static: file tracked on `main` HEAD, present in all four tags, and present in ~26 of 31 remote branch heads (grep over remote refs). Purge must therefore cover branches and tags, not just `main` history.

04-SECURITY/RBAC.md

Role-Based Access Control (RBAC)

Status: current (VERIFIED: no roles) + proposed native matrix. Commit `2dd2fe4`.

1. Current reality (VERIFIED)

The current app has **no roles and no users**: it is a single local-user product. There is no Owner/Driver/Labourer distinction in code, no auth, no permission model. Roles exist **only as data fields**: a Trip stores a `driverName` string; a Labourer is a `Labour` entity + per-trip `isPresent`. There is no account-driven differentiation.

The brief's Owner is named **Ramesh Sahu**; that name is absent from the codebase. Representing the owner must be a **backend-authoritative** role in native, never hardcoded credentials.

2. Current capability table (VERIFIED)

Capability	Current (single implicit app user)
View dashboard	Yes (S-02)
Manage users	No such concept
Assign jobs	N/A (trip data entry only)
View assigned jobs	View own trips only
Modify job/trip status	Edit trip details/work
Attendance	Toggle present/absent
Payments	No
Notifications	No
Reports	Light analytics only (S-05)
System settings	Backup/restore only (S-06); theme/about inert
Audit logs	None

3. Proposed RBAC matrix (PROPOSED — native, server-enforced)

Legend: □ allowed; □ owner only (server-gated); □ denied; (—) not applicable. Enforcement = backend rules/Cloud Functions; UI simply hides items.

Capability	Owner/Admin	Driver	Labourer/Worker
View dashboard	□ (full)	□ (driver home)	□ (own attendance)
Manage users (add/deactivate/suspend)	□	□	□
Assign jobs/works	□	□	□
View assigned jobs	□ (all)	□ (own)	□ (attendance only)
Modify job status	□	□→(own trip status PROPOSED review)	□
Record/update attendance	□/□	own confirmation only	view own
Payments/wages	□ (owner)	□ (view if provided)	view own if provided
Notifications send	□	receive own	receive own
Reports	□	□ (limited own summary)	□
System settings/config	□	□	□

Capability	Owner/Admin	Driver	Labourer/Worker
Audit logs			
Profile management	own	own	own
Account/role management			
Suspension/deactivation			
Data export			own data request
Deep-link/notif handling		own	own

Notes: - "Owner/Admin " means only an authenticated user whose server role is owner/admin may perform it. - Any capability that a role may perform on its own records must still be scoped in rules by `ownerId` / membership (IDOR protection). - Do not assume all roles have identical access; do not grant labourers/ drivers admin abilities.

4. Enforcement model

- Auth UID → server role (custom claims / role doc).
- Firestore rules + Storage rules check `request.auth` role & ownership.
- Cloud Functions perform privileged ops & role assignment (admin-only).
- UI role flags = display/routing only.

See `AUTHENTICATION-AUTHORIZATION.md` , `THREAT-MODEL.md` , `08-NATIVE-ANDROID/FIREBASE-SECURITY-RULES.md` .

04-SECURITY/PRIVACY-DATA-FLOW.md

Privacy & Data Flow

Status: current (VERIFIED) + native (PROPOSED). Commit `2dd2fe4` .

1. Personal data inventory (current, VERIFIED)

Data	Type	Where	Sensitivity
Labourer name	String	`labour_box` + backups	Personal
Labourer phone (optional)	String?	model exists; **no UI captures it**	Personal (contact)
Driver name	String (on Trip)	`trip_box` + backups	Personal
Work/trip metadata (date, place, notes, times)	String/ DateTime	boxes + backups	Business/ operational
No geolocation, no DOB, no government IDs, no images of people, no payment data	—	—	N/A

2. Data flows (current)

```
UI ──> Bloc ──> Usecase ──> Repository ──> Hive (device sandbox)
Settings ─> export .labourbackup (plaintext JSON) ─> user-chosen storage (SAF)
Settings <─ pick .labourbackup <─ restore → Hive (snapshot+rollback)
```

No data leaves the device via network. The only external flow is the user-exported backup file and the app-support directory contents if the device is backed up/migrated by OS tooling.

3. Privacy risks (current)

- PR-1 (MED): Plaintext PII in Hive and backups; unencrypted.
- PR-2 (LOW): Restore file could contain arbitrary records (data authenticity).
- PR-3 (INFO): No privacy policy/consent mechanism or data-deletion UX (single offline device; deletion = uninstall/clear data or full restore).
- PR-4 (INFO): Error messages embed raw exception text (minor data leakage in logs).

4. Proposed native data flow (PROPOSED)

```
Owner/Driver/Labourer (App) --> App Check --> Firebase Auth (UID)
   --> Firestore/Storage (rules by role+owner) --> Cloud Functions (authz/validation)
Audit log (backend-only writes); FCM (role-scoped) via Cloud Functions.
Offline queue on device --> reconcile on reconnect (server-validated).
```

- Data minimization: collect only required fields; do not add biometrics/excess PII.
- Retention & deletion: define per-data-class retention; user/account deletion path that cascades via Cloud Functions respecting business/regulatory holds.
- Consent/privacy notice before enabling optional analytics.
- Encryption at rest for local PII; authenticated cloud backup.

5. Required privacy artifacts (native)

Privacy policy, data-processing notes for operators, consent for notifications/analytics, deletion & export (account data portability) flows. Marked PROPOSED/not-yet-produced.

6. Verification status

VERIFIED current inventory & flows (no network). Native privacy program is PROPOSED and absent today.

08-NATIVE-ANDROID/FIREBASE-ARCHITECTURE.md

Firebase Backend Architecture (recommended)

Status: PROPOSED design spec — no Firebase exists in the current repo (VERIFIED absence). Commit audited `2dd2fe4`.

1. Service decisions

Service	Use	Rationale / notes
Firebase Authentication	identity & session	email/password primary; phone evaluated for operator/driver context
Cloud Firestore	domain data (users, roles, jobs/works, trips, attendance, notifications, audit)	flexible, offline persistence, rules-based security
Cloud Storage	profile images / approved media / backup files	rules-protected
Cloud Functions	trusted ops: role assignment, privileged/transactional workflows, notifications, validation, scheduled jobs, aggregation	server-authoritative boundary
Firebase App Check	protect backend from unauthorized clients	enable
Crashlytics	production crash monitoring	enable
Analytics	only if real value + privacy-compliant opt-in	optional
Remote Config	only if a remotely configurable behaviour is genuinely useful	optional

Do not add Firebase products to look sophisticated.

2. Trust model

Client never authorizes itself. Backend (rules + Functions) authorizes every sensitive read/write. Role flags on client are for display only.

3. Reference data flow (owner adds a trip / job)

```
Owner (Compose) → ViewModel → SaveJobUseCase → repository.saveJob(job)
   → (server) rules check: request.auth.uid has role owner; doc write valid
   → optional Cloud Function onWrite for side-effects (notify driver, aggregate, audit)
```

4. Offline-first strategy

- Firestore local persistence enabled; local writes queue and reconcile.
- Domain invariants (e.g., trip numbering) enforced server-side to avoid client divergence; conflict policy defined (no silent overwrite).
- WorkManager retries with backoff.

5. Reference architecture diagram

```
flowchart LR
  Owner[Owner App] & Driver[Driver App] & Lab[Labourer App]
  --> AppCheck[App Check] --> Auth[Firebase Auth]
  Auth --> FS[Firestore rules]
  Auth --> ST[Storage rules]
  CF[Cloud Functions: roles/notif/validate/audit] --> FS
  Owner -. privileged .-> CF
  FS --> Push[FCM + CF]
  Push --> Driver & Lab
  CF --> Crashlytics
```

6. Security considerations

See `04-SECURITY/SECURITY.md`, `FIREBASE-SECURITY-RULES.md`, `THREAT-MODEL.md`, `RBAC.md`.

7. Ownership note

The owner account (brief: Ramesh Sahu) is provisioned server-side (role bound to UID via Cloud Function at onboarding). No hardcoded credentials. See `AUTHENTICATION-AUTHORIZATION.md`.

08-NATIVE-ANDROID/FIREBASE-DATABASE.md

Firestore Database Design (recommended)

Status: PROPOSED design spec. Fields marked "proposed" are recommendations; nothing exists in the current repo. Commit audited `2dd2fe4`. Map each current Hive collection (`DATABASE.md`) to a Firestore collection while adding ownership/audit.

Conventions

- IDs: Firestore auto-IDs or deterministic per domain; store `createdAt`, `updatedAt` (server timestamps where possible), `ownerId` (the org/owner), `createdBy` / `changedBy` (auth.uid), soft-delete flag `deleted:boolean` where needed.
- No client-trusted role fields as sole authorization; roles in user doc + custom claims.
- Data minimisation: only fields actually required.

Proposed collections (each with required/optional/ownership/rules/audit)

`organizations` (orgs/tenants)

Purpose: top-level owner scope. Owner: org admin. Read/write: members by role (rules). Required: name, ownerUid, createdAt. Optional: settings. Indexes: by ownerUid. Security: only owner/admin manage; members read. Audit: on config change.

`users` / `profiles`

Purpose: identity+role+profile. Readable by: self (full), others (limited), admin. Writable by: self (limited profile), admin/CF (role/status). Required: uid, role, status(active/suspended), displayName, createdAt. Optional: phone, profileImage, orgId. Indexes: by orgId+role, by status. Audit: role/status changes.

`jobs` / `works`

Purpose: a unit of work (mirrors `Work`, plus optional assignment to a driver & job status). Owner: organization/owner. Readable by: members by role (owner all; driver assigned; labourer if linked). Writable by: owner (create/update status via rules/CF); driver limited status. Required: orgId, workType, session, date, assignedDriverId?, status, place, createdBy. Optional: notes. Indexes: orgId+date+status; orgId+assignedDriverId+status. Audit: status & assignment transitions.

`trips` (optional if job-centric; or fold into jobs)

If kept as separate child: each `trip` under a job/work (subcollection `jobs/{id}/trips`). Required: tripNumber, tractorId, driverId, jobId, status, createdBy. Optional: notes, place. Indexes: jobId+tripNumber. Audit: status changes.

`tractors` / `vehicles`

Optional master list owned by org. Required: name/plate. Audit on edits.

`labourers` / `crew`

Mirrors `Labour` master; add orgId, active, wage details (owner-only). Optional: phone, dailyWage, bank (if payments). Indexes: orgId+active. Audit: wage/status changes.

`attendance`

Per worker per date/trip. Required: orgId, labourerId, date, session, tripId, status(present/absent), recordedBy. Writable: owner/admin and (with authorisation) authorised input; labourer read own. Indexes: labourerId+date; tripId. Audit: corrections by admin only. (Mirrors `TripLabour`.)

`notifications`

System/user notification history. Required: userId, type, title/body ref, read, createdAt, deepLink. Indexes: userId+read+createdAt. Writable: server/CF only. Audit on send.

`auditLog`

Admin/privileged action log. Required: actorUid, action, targetType, targetId, timestamp, metadata. Writable: server/CF only. Retention: policy-defined.

`backups` / media (Storage)

Purpose: authenticated export/media. Rules per role; owner writes/reads; App Check.

Relations summary

org → users; org → jobs → trips → attendance; org → labourers; users(role) authorize reads/writes by rules. Mirrors current model but with owner scope & audit.

Consistency & transactions

- Use batched writes/ `runTransaction` for multi-doc invariants (trip numbering, attendance + job counters, role+profile updates).
- Denormalise only where query patterns justify (e.g., labourer displayName on attendance) with documented propagation; avoid over-denormalization.

Retention / lifecycle

- Soft-delete for audit & undo; hard-delete only server-side with hold checks.
- Retention policy per collection (audit, notifications).

Mapping from current Hive

Hive → Firestore: `work_box` → `jobs/works` (+org); `trip_box` → `jobs/{id}/trips` ; `labour_box` → `labourers` ; `trip_labour_box` → `attendance` ; `draft_box` → local draft only (Room/DataStore, not Firestore).

Findings (from current product) that shape design

- No auth/ownership today → add orgId + createdBy everywhere.
- Attendance currently loses history (latest state only) → design versioned/audited attendance.
- Deterministic work-id ambiguity → use server timestamps/auto-ids.
- Trip `status` unused → define an explicit job/trip status lifecycle.

Verification status

PROPOSED only; no current Firestore database exists (VERIFIED absence in repo).

08-NATIVE-ANDROID/FIREBASE-SECURITY-RULES.md

Firebase Security Rules (recommended)

Status: PROPOSED (rules pseudocode/design only; no Firebase in current repo). Commit audited `2dd2fe4`.

1. Principles

- Default-deny. Every rule checks `request.auth != null`, role, and ownership.
- Client role flags never authorize; rules/CF re-check using `request.auth.token.role` and user/role docs.
- Use custom claims for coarse role checks in rules; role documents for richer metadata.
- Cloud Functions for operations rules cannot express (transactions, aggregation, role assignment, notifications).

2. Helper functions (pseudocode)

```
function signedIn() { return request.auth != null; }
function isOwner(orgId) {
  return request.auth.token.org == orgId && request.auth.token.role in ['owner','admin'];
}
function isMember(orgId) { return request.auth.token.org == orgId; }
```

3. Firestore rules sketch

```
rules_version = '2';
service cloud.firestore {
  match /databases/{db}/documents {
    function signedIn() { return request.auth != null; }
    function role() { return request.auth.token.role; }
    function orgOf() { return request.auth.token.org; }

    // profiles: users may read self; owner may read org users
    match /users/{uid} {
      allow read: if signedIn() && (uid == request.auth.uid || role() in ['owner','admin']);
      allow create: if signedIn() && uid == request.auth.uid;
      allow update: if signedIn() && uid == request.auth.uid
                    &&
request.resource.data.diff(resource.data).affectedKeys().hasOnly(['phone','displayName','profileIm
age']);
      // role/status changes ONLY via Cloud Function (admin)
    }

    // jobs
    match /jobs/{jobId} {
      allow read: if signedIn() && isMember(resource.data.orgId)
                  && (role() in ['owner','admin','manager']
                      || resource.data.assignedDriverId == request.auth.uid);
      allow create: if signedIn() && role() in ['owner','admin']
                    && request.resource.data.orgId == orgOf();
      allow update: if signedIn() && role() in ['owner','admin'] &&
orgOf()==request.resource.data.orgId;
      // driver may only transition own job status via Cloud Function (not direct write)
      // nested trips/attendance similar with org + ownership checks
    }

    // auditLog — server only
    match /auditLog/{id} {
      allow read: if signedIn() && role() in ['owner','admin'];
      allow write: if false;   // only Cloud Functions (admin SDK bypasses rules)
    }
  }
}
```

4. Storage rules sketch

```
service firebase.storage {
  match /b/{bucket}/o {
    match /orgs/{orgId}/profile/{uid}/{file} {
      allow read: if signedIn() && member(orgId);
      allow write: if signedIn() && (uid == request.auth.uid && contentType within allowlist)
                   || role in owner/admin;
    }
    match /backups/{orgId}/{file} {
      allow read,write: if signedIn() && role() in ['owner','admin'] && orgOf()==orgId;
    }
  }
}
```

5. Cloud Functions authorization rules

- Role assignment & change: admin-only function; verifies caller is owner/admin via admin SDK; writes profile + updates custom claims; audit logs.
- Job assignment / status transitions: validates role & state machine; writes atomically; notifies via FCM; audit logs.
- Notifications & scheduled jobs: CF only.
- Backup creation/restore: CF (owner) writes to Storage with App Check + auth.

6. App Check

- Enforce App Check on Firestore/Storage/Functions so only the genuine app reaches resources; handle Play integrity/attestation provider per platform.

7. Replay / abuse / rate limiting

- Cloud Functions enforce quotas; idempotency keys for retried operations; alert on anomaly.

8. Testing the rules

- Use the Firebase emulator + rules unit tests in CI to validate role matrix (owner/driver/labourer × read/write per collection).

9. Do-not

- Never write rules allowing `allow write: if true`, broad `get('/users/..')` without role checks, or client-side role trust.

PERFORMANCE

05-PERFORMANCE/PERFORMANCE.md

Performance (current) — overview

Status: analysis VERIFIED where possible; runtime numbers `UNVERIFIED` (no Flutter/device in sandbox; benchmarks present in repo but not re-run here). Commit `2dd2fe4`.

Current posture

- Offline, single-user, Hive-backed. Realistic data volumes are small-to-moderate, but long history growth and many labours per trip are the main scaling concerns.
- The code shows deliberate performance intent for `TripLabour`: composite keys `"<tripId>_<id>"`, prefix-scan reads, and `putAll` bulk writes (`work_local_data_source.dart`).
- The repository contains benchmark/perf tests under `test/performance/` and `test/integration/benchmark_trip_labour_insert.dart`, indicating earlier measurement work by the authoring agent.

Benchmark claims in repo (VERIFIED existence, numeric results NOT re-run here)

- `test/performance/trip_labour_loading_benchmark_test.dart`, `trip_labour_write_benchmark_test.dart`, `test/features/work/data/datasources/performance_benchmark_test.dart`, `test/integration/benchmark_trip_labour_insert.dart`.
- Because the Flutter SDK is unavailable in this environment, these were **not re-executed**; their pass/numbers are `UNVERIFIED`.

Known risk areas (VERIFIED statically)

Area	Concern
Full-box scans	`getWorks`, `getAllTrips`, `getLabours`, `getLaboursForTrips` iterate `box.values`
Name resolution	`trip_details_screen.dart` uses `firstWhere` over the full labour list per labour row → O(trips×labours) worst case
Backup serialize	Reads entire DB and JSON-encodes all rows (memory spike, esp. near 25 MB)
Fonts	`google_fonts` runtime network fetch can block/delay glyph rendering offline
Nested scrolling	Dashboard `ListView` with nested non-scroll lists; `ListView.builder` inside it
Database growth	No retention/compact strategy for Hive; delete leaves tombstones

Native (PROPOSED) performance priorities

Stable Compose state; `LazyColumn` keys; Coil image caching; paging for large lists; coroutine dispatchers (IO for DB/network); baseline profiles; R8; startup & macrobenchmarks; memory profiling. See `PERFORMANCE-OPPORTUNITIES.md`.

05-PERFORMANCE/PERFORMANCE-AUDIT.md

Performance Audit (current)

Status: VERIFIED static; numeric UNVERIFIED. Commit `2dd2fe4`. Severity for performance risks.

Findings

ID	Sev	Category	Finding	Evidence	Mitigation
PERF-1	MED	DB	Full-value scans for whole collections (`getWorks` , `getAllTrips` , `getLabours` , `getLaboursForTrips`)	data source methods	Index by key date; batch; move to indexed store (native)
PERF-2	MED	UI	Labour-name resolution scans full labour list per labour row (`firstWhere`)	trip_details	Build id→name map once; native: indexed join
PERF-3	LOW	Data	Backup serialises entire DB in memory & writes large JSON	settings	Stream; cap; native background via WorkManager
PERF-4	LOW	Font	`google_fonts` network fetch	app_theme	Bundle fonts
PERF-5	LOW	UI	Loop/shimmer animations on FAB & counters (`repeat`)	dashboard	Reduce decoration; honour reduced motion
PERF-6	INFO	DB	Hive compaction/tombstones on deletes unmanaged	—	periodic compact/repack
PERF-7	INFO	Blocking	UI-triggered awaits on many sequential box writes (e.g., full-trip save loops)	bloc save handlers	Use batch `putAll` ; native DB transactions

Verified good practice (VERIFIED)

- TripLabour composite-key prefix reads + `putAll` bulk writes (PERF intent present).
- Skeleton loading improves perceived startup.
- Benchmark/perf tests exist in the repo.

Gaps in evidence

- No cold-start/session-time numbers measured in this environment. Prior repo perf reports exist under `docs/production_readiness/performance_report.md` but were **not** independently reproduced (`UNVERIFIED`).

Verification status

VERIFIED structural. Numeric/perf thresholds UNVERIFIED without device+SDK.

05-PERFORMANCE/STARTUP-ANALYSIS.md

Startup Analysis (current)

Status: VERIFIED (static flow); timing `UNVERIFIED`. Commit `2dd2fe4`.

Startup sequence (VERIFIED)

1. OS launches `MainActivity` (`FlutterActivity`) with launch theme → Flutter engine boots.
2. `main()` runs: `WidgetsFlutterBinding.ensureInitialized()` , `HiveSetup.init()` (registers 5 adapters + opens 5 boxes on the app-support filesystem), `di.init()` (get_it), then `runApp` .
3. `/splash` shows for ~1.5 s (a fixed delay), then `/dashboard` .
4. Dashboard dispatches `LoadDashboardDataEvent` ; shows skeleton while loading.

Observations

- **Fixed 1.5 s splash** adds latency regardless of readiness (it is decorative, not a load-gate). Could gate on first-frame/data-ready instead.
- Hive opens 5 boxes synchronously awaited in `main()` before `runApp` — can add to cold start on slow storage; acceptable at single-user scale.
- No heavy image assets (only a 192px icon). Fonts may fetch over network (if used at startup) delaying text style resolution.

Native (PROPOSED) targets

- Cold start ≤ 2 s on mid-range device (target, not current claim).
- Firebase initialised lazily; baseline profile; minimal startup work off main thread; skip decorative splash or make it a proper data-gate.

Verification status

VERIFIED startup structure. Measured cold-start time UNVERIFIED (no device in environment).

05-PERFORMANCE/NETWORK-ANALYSIS.md

Network Analysis (current)

Status: VERIFIED. Commit `2dd2fe4` .

No network usage

The current product performs **no network I/O**: - No `INTERNET` permission in `AndroidManifest.xml` . - No HTTP/WebSocket/grpc packages (no `http` , `dio` , `web_socket_channel`). - No remote calls, no backend, no CDN; no Firebase.

The only "external I/O" is local file access via `file_picker` /SAF for backup/restore (local to the device/user-chosen storage).

Implication

- There are no network-latency, retry, timeout, or bandwidth concerns today. "Offline" is not a degraded mode — it is the only mode.
- When the native product introduces Firebase, a real network/latency/offline analysis becomes necessary (see `08-NATIVE-ANDROID/` and `THREAT-MODEL.md`). Offline-first with online reconciliation is PROPOSED.

Verification status

VERIFIED (no networking stack present).

06-QUALITY/TESTING.md

Testing (current) — overview

Status: VERIFIED presence & scope; execution UNVERIFIED (no Flutter SDK in environment). Commit `2dd2fe4`.

Current test suite (VERIFIED files)

Area	Files	Coverage intent
Unit — blocs	`work_bloc_test`, `history_bloc_test`, `labour_persistence_test`, `search_filter_test`	state transitions, search/filter, labour persistence
Unit — usecases	`trip_numbering_test`, `date_partition_test`, `get_works_usecase_test`	business rules
Unit — core	`failures_test`, `date_time_utils_test`	utilities
Unit — settings	`restore_logic_test`	backup parsing/ validation rules
Widget	`dashboard_screen_test`, `dashboard_regression_test`, `dashboard_state_test`, `trip_edit_test`, `trip_details_screen_test`, `history_screen_test`, `analytics_screen_test`, `empty_state_widget_test`	per-screen behaviour
Integration	`continuity_test`, `edit_flow_test`, `final_manual_qa_simulation_test`, `benchmark_trip_labour_insert`	data integrity & flows
Performance	`trip_labour_loading_benchmark_test`, `trip_labour_write_benchmark_test`	load/write benchmarks
E2E/QA gate	`e2e/final_qa_gate_test.dart`	holistic gate
Helpers	`helpers/mock_work_repository.dart`	mock repo (test-only)

Total: 27 test files, ~2,505 lines.

Important honesty note (VERIFIED, no fake testing)

- Mocks are confined to tests (`mock_work_repository.dart`); production uses real Hive. No fake/ dummy production backend, no hardcoded accounts. Good.
- **BUT** `restore_logic_test.dart` states it tests validation rules indirectly (writes files & re-checks rules) because the parse logic is a private top-level function — not a true unit test of the function. Coverage gap.

Execution status

This environment has **no Flutter/Dart SDK**, so `flutter test` could not be re-run; pass/fail state and any counts are `UNVERIFIED` here. Prior agent docs claim a passing gate; not independently reproduced.

Native test strategy (PROPOSED)

Unit (ViewModel/usecase/repository), Firebase integration (emulator suite), Compose UI tests (create/edit/delete/attendance/search/history), navigation tests, accessibility tests, offline/error/RBAC/security tests, performance (macrobench), release tests. Critical user journeys must have automated coverage. See `08-NATIVE-ANDROID/`.

06-QUALITY/TESTING-AUDIT.md

Testing Audit (current)

Status: VERIFIED (static scope); execution UNVERIFIED. Commit `2dd2fe4`.

Findings

ID	Sev	Finding	Evidence
T-1	MED	Execution of the suite not independently reproduced (no SDK in env); pass state UNVERIFIED	environment
T-2	MED	`restore_logic_test` does not unit-test the private `_parseAndValidateBackup` function directly; tests re-implement rules	restore_logic_test.dart header comment
T-3	MED	No golden/screenshot tests; regression relies on widget tests	repo
T-4	LOW	No coverage report in repo root; coverage only mentioned in prior docs	docs/production_readiness/coverage_report.md (not verified)
T-5	MED	No accessibility tests, no security tests, no RTL/locale tests	repo
T-6	LOW	Performance tests exist but are microbenchmarks; no macrobenchmark/startup/jank test	test/performance
T-7	MED	No error/offline/retry-path UI tests (though some error states are rendered)	test/widget
T-8	INFO	Positive: unit/widget/integration split is sensible; mocks isolated in test helpers	structure

Test hygiene (VERIFIED)

- Mocks only inside tests. No placeholder/dummy production backend. Good.

Gaps to close for native

Per `TESTING.md`. Emphasise: critical journey coverage (create trip → attendance → history), error/offline, RBAC/security, accessibility, and macrobenchmark.

Verification status

Static audit VERIFIED. Any "all tests pass" claim UNVERIFIED in this environment.

Test Coverage Map (current)

Status: VERIFIED file map; actual line coverage UNVERIFIED (not measured here). Commit `2dd2fe4`.

Feature → test mapping

Feature/rule	Covered by test(s)	Notes
Trip numbering (morning/ evening/new date)	`unit/usecases/trip_numbering_test`	VERIFIED presence
Date partition/isolation	`unit/usecases/date_partition_test`	
GetWorks	`unit/usecases/get_works_usecase_test`	
WorkBloc transitions	`unit/blocs/work_bloc_test`, `widget/dashboard_state_test`	
Search/filter (bloc)	`unit/blocs/search_filter_test`	filter predicates not implemented; test likely search only
Labour persistence	`unit/blocs/labour_persistence_test`	
History bloc	`unit/blocs/history_bloc_test`	
DateTime utils / failures	`unit/core/*`	
Backup/restore validation	`unit/settings/restore_logic_test`	indirect (see T-2)
Dashboard UI	`widget/dashboard_screen_test`, `dashboard_regression_test`, `trip_edit_test`	
Trip details UI	`widget/details/trip_details_screen_test`	
History UI	`widget/history/history_screen_test`	
Analytics UI	`widget/analytics/analytics_screen_test`	
Shared empty-state	`widget/shared/empty_state_widget_test`	
Continuity & edit flow	`integration/continuity_test`, `edit_flow_test`	
Manual QA sim / gate	`integration/final_manual_qa_simulation_test`, `e2e/final_qa_gate_test`	
Performance	`performance/*`, `integration/benchmark_trip_labour_insert`	microbench

Not covered (VERIFIED absence)

- Settings Backup UI flow (widget) & the full restore transaction (rollback path) — only validation logic indirectly tested.
- Settings screen widget test missing.
- Details screen (orphaned) no dedicated widget test (dead-code branch implies not covered).
- AddEditWorkScreen draft autosave/restore UI test.
- ConfirmNextTripScreen save flow UI test.
- Error/retry UI states.
- Accessibility, localization, security tests — none.

Native target

Compose UI + Espresso mapped 1:1 to journeys; see `08-NATIVE-ANDROID/`.

CI/CD

07-OPERATIONS/CI-CD.md

CI/CD

Status: current (VERIFIED: none in repo) + native pipeline (PROPOSED). Commit `2dd2fe4`.

Current

- **No CI/CD configuration** in the repository (no `.github/workflows`, no `.gitlab-ci.yml`, etc.).
- Development commands are documented (`flutter pub get`, `analyze`, `test`, `build apk --release`).
- Prior RC builds appear to have been performed manually/agent-side; release artifacts and signing rely on local `key.properties`.

Current gaps

- No automated quality gate, no secret scan, no artifact verification pipeline.
- No build matrix for signing/versioning.

Native CI/CD (PROPOSED)

Pipeline (with production secrets from secure CI secret storage — never commit credentials):

```
checkout
→ dependency validation
→ static analysis (Detekt/lint)
→ format check (ktlint)
→ unit tests
→ integration tests (incl. Firebase emulator)
→ Compose UI tests
→ security checks (gitleaks/SAST; dependency audit)
→ release build (debug + release)
→ artifact verification
→ optional Firebase/Play deployment (staged)
→ release approval
```

- Secrets in CI secret store: upload keystore, keystore passwords, Google Play service account, Firebase config.
- App Check, Crashlytics enabled for release.

Verification status

VERIFIED absence of current CI. Pipeline is PROPOSED.

07-OPERATIONS/DEPLOYMENT.md

Deployment

Status: current (VERIFIED process from repo/docs) + native (PROPOSED). Commit `2dd2fe4`.

Current release engineering (VERIFIED)

- Build: `flutter build apk --release` requires `android/key.properties` (release signing). Release build type: `minifyEnabled=true`, `shrinkResources=true`, proguard file present (`android/app/proguard-rules.pro`).
- Application id: `com.roshan.labourparty`; version `1.0.0+1` (from pubspec).
- Signing: keystore config via `key.properties` (not committed); only `key.properties.example` committed. **Keystore-backup file is wrongly committed (BUG-01).**
- Distribution: prior release candidates tagged (`v1.0.0`, `v1.0.0-rc1`, `v1.0.1-hotfix-rc1`, `RC-1.3`). No in-repo evidence of a specific Play console/app-store deployment record (see CODEBASE note; `UNVERIFIED` whether app is actually published).

Release process expectations (from docs, VERIFIED content)

See `docs/RELEASE.md`: generate `upload-keystore.jks` (RSA 2048+, long validity), keep secret, create `key.properties`, build release. Data migration between mismatched signature installs requires `.labourbackup` export/restore (README).

Native (PROPOSED) — release engineering

- Pin versioning (versionCode/versionName) with automation.
- Use **Play App Signing**: upload key kept in CI secret store / Google Play; app signing key managed by Play.
- Build variants: debug vs release (and flavours if roles/regions differ).
- R8 resource shrinking with tested rules; proguard kept.
- Staged rollout (e.g., 10% → 100%) with monitoring; rollback via prior artifact.
- Release notes + changelog; post-release verification & crash monitoring.

Verification status

VERIFIED build/signing config. Whether a Play-store release exists `UNVERIFIED`.

08-NATIVE-ANDROID/ANDROID-REBUILD-PRD.md

Native Android Rebuild — Product Requirements (draft)

Status: PROPOSED (draft for the future native product). No native app exists yet. It supersedes/augments the current offline single-user product per the migration brief. Commit audited `2dd2fe4`.

1. Positioning

Rebuild "Labour Party" as a native Android app (Kotlin + Jetpack Compose + Material 3) that preserves the **validated core business behaviour** of the current offline product while adding secure multi-role operation and an optional backend. Offline-first remains a priority (field usage).

2. Retain from current product (VERIFIED behaviour to carry over)

- Core domain: Work/Session, Trips, Labour, per-trip attendance.
- Sequential trip numbering across morning/evening and per date.
- Data integrity on edit/delete/restore (no orphans; cascade delete; soft-mark removed attendance).
- Quick "next trip" copy from the previous trip.
- Grouped history + analytics; search.
- Local backup/restore as a portable export.
- Draft autosave to prevent data loss.
- Field-oriented, fast data entry (chips for tractor, quick attendance toggles).

3. Change / improve (PROPOSED)

- Add secure authentication and explicit roles (Owner/Admin, Driver, Labourer where justified) with **server-authoritative** authorization.
- Add optional Firebase backend (Auth, Firestore, Storage, Cloud Functions, FCM) while retaining offline-first local behaviour.
- Fix known defects: signed/encrypted backups, retry on errors, accessibility, contrast, localization-ready strings, adaptive/tablet layouts, proper theme/light mode, no orphaned routes or inert controls.
- Reports & payment ledger only if product need is confirmed.

4. Explicit non-objectives for v1 native (PROPOSED)

Do not over-engineer: no web/desktop unless required; no analytics/advertising; no biometric PII; keep dependencies justified.

5. Roles

See `04-SECURITY/RBAC.md`. Owner "Ramesh Sahu" identity must be represented server-side at provisioning, **never hardcoded**. Resolve branding/package identity (OQ-2).

6. Acceptance themes

- Role-gated journeys work end-to-end (owner assigns → driver completes → owner reports).
- Offline entries queue and reconcile without data loss.
- All critical-journey tests pass; security rules tested; release gate (PRE-RELEASE.md) passes.

7. Success metrics (targets — not measured)

Crash-free ≥99.5%; cold start ≤2 s; data entry fast; zero silent data-loss; successful offline→online reconcile 100% on tested paths.

8. Open questions

OQ-1..OQ-8 (see product/security docs). Everything marked PROPOSED pending confirmation.

08-NATIVE-ANDROID/ANDROID-ARCHITECTURE.md

Native Android Architecture (recommended)

Status: PROPOSED design spec — not implemented. Commit audited `2dd2fe4`.

1. Stack (baseline)

Kotlin + Jetpack Compose + Material 3 + AndroidX Navigation + ViewModel + StateFlow/Flow (UDF/MVI-style) + Clean Architecture + Hilt + Coroutines/Flow + Repository pattern + Room (only where local relational persistence is justified) + DataStore (prefs) + Coil (images) + WorkManager + Firebase (Auth, Firestore, Storage, Cloud Functions, FCM, App Check, Crashlytics; Analytics/Remote Config only where justified).

2. Module view (only if justified; avoid over-modularization)

```
:app                        (Compose entry, DI graph)
:core:designsystem          (theme, components)
:core:common                (utils, dispatchers, result types)
:feature:auth
:feature:work / jobs        (owner trip/job + driver view)
:feature:crew               (labourer & driver management)
:feature:attendance
:feature:reports
:feature:settings           (backup, profile, account)
:data:local  (Room/DataStore)   :data:remote (Firebase datasources)
:domain                     (entities, usecases, repository interfaces)
```

For an MVP, a **single :app with clear package layering** is acceptable; split only as size/team require.

3. Layering & dependency direction

```
Presentation (Compose UI + ViewModel)
        │
        ▼
Domain (entities, usecases, repository interfaces)   <- business rules, role-agnostic
        │
```

```
      ▼
Data (Room/local + Firebase/remote repositories & datasources, DTO mapping)
```

Rules: - UI never calls Firebase/Room/business logic directly — goes through ViewModel → UseCase → Repository. - Domain independent of Android & Firebase. - Repository interface is the seam; local/remote implementations swapped/tested. - DTO↔domain mapping at data boundary.

4. State architecture (UDF/MVI)

```
[User Action]
    │
    ▼
ViewModel (events/Intents → reduce to state)
    │
    ▼
UseCase → Repository → Firebase / local data
    │
    ▼
Result → StateFlow<UiState> (sealed)
    │
    ▼
Compose (collectAsStateWithLifecycle)
```

Explicit states: Loading / Success / Empty / Error / Offline / Unauthorized / Forbidden. One-way events for navigation/one-shot (snackbar) via a separate channel if needed (avoid navigation-as-state). No arbitrary mutable global state.

5. Roles & authorization seam

- ViewModel takes the authenticated role only to decide which **screens/actions to show**.
- Every privileged path still calls a server-enforced operation (Firestore rules / Cloud Functions).
- Use a `RoleProvider`/auth repository returning `AuthUser{uid, role}` observed via Flow.

6. Persistence

- Local relational: Room where joins/cascade/offline-querying justify it (trips, attendance, drafts, offline queue).
- Preferences: DataStore.
- Images: Coil + cache; approved media in Cloud Storage.

7. Background work

- WorkManager for scheduled backups, offline-queue replay, notification-triggered refresh.
- Never rely on UI lifetime for critical writes.

8. Concurrency

- Coroutines; `Dispatchers.IO` for DB/network; main-safe ViewModels; `StateFlow` + `conflate` where appropriate; structured concurrency via Hilt-scoped coroutines.

9. Quality

- Stable `@Stable` UI state, keys on `LazyColumn`, baseline profiles, R8 rules, macrobenchmarks, LeakCanary in QA.

10. Anti-goals

No business/backend logic in composables; no client-side-only authorization; no global mutable state; no synchronous work on main thread; no network on main.

See also `STATE-MANAGEMENT.md`, `CURRENT-ARCHITECTURE.md`, and the migration matrix.

08-NATIVE-ANDROID/ANDROID-DESIGN-SYSTEM.md

Native Android Design System (specification)

Status: PROPOSED design spec (Jetpack Compose + Material 3). Commit audited `2dd2fe4`. Visual target: premium, modern, polished, consumer-grade (no adult content; premium refers to hierarchy, media, cards, CTAs, motion, discovery).

1. Brand & identity

- Brand: "Labour Party" (resolve identity/package with owner — OQ-2).
- Logo: current `app_icon_192.png` as source; create adaptive launcher icon + in-app mark.
- Iconography: Material Symbols/Outlined consistent set; brand-only accents.

2. Color system

- Build from brand seed (current primary `#246BFD`-like blue → cyan accent `#00D2FF`) via `dynamicLightColorScheme/dynamicDarkColorScheme` or fixed `MaterialTheme`.
- Semantic: success (present/attendance), error, warning, info mapped to M3 containers.
- Dark + light schemes; optional dynamic color (from wallpaper) behind a setting.
- Provide contrast-AA variants; never rely on alpha-38/54 for body text.

3. Typography

- Bundle fonts (Poppins for display/titles + Inter or similar for body) — no runtime fetch.
- M3 type scale (display/headline/title/body/label) mapped; line-height & tracking sane; supports large font scale.

4. Shape / spacing / elevation

- Shape scale: small 8 / medium 12 / large 16 / extra-large 24 (cards 16/24).
- Spacing: 4/8/12/16/24/32 system; page gutters 16.
- Elevation via M3 tonal + shadow tokens; elevation is semantic not decorative.

5. Core components

- Surfaces/cards (Card, tonal/elevated/outlined), Buttons (Filled/Outlined/Tonal/Text), TextFields (Outlined, filled), Dialogs & Sheets (bottom sheets for filters), TopBar, BottomBar + NavigationRail (tablet), Chips, Badges, Lists (LazyColumn w/ keys), Loading (Linear/Circular + skeletons), Empty/Error states with retry, Snackbars, Swipe/Undo actions.

6. Motion & transitions

- Material motion tokens; shared-axis for screens; content fade/scale; keep subtle; honor `reduced motion`.

7. Responsive / large screen

- Phone canonical; width-based adaptive: use `BoxWithConstraints` / `MaterialWindowSizeClass`; NavigationRail + master/detail on tablets; multi-pane owner dashboard where valuable.

8. Accessibility

- Semantics everywhere, 48dp targets, AA contrast, large-font safe, reduced motion, TalkBack announcements (see ACCESSIBILITY.md).

9. Theming file layout (Compose)

`designsystem/theme/{Color,Type,Shape,Elevation}.kt`, `Theme.kt` (light/dark/dynamic), reusable component composables in `designsystem/components`.

10. Do-not

- No sexual/adult content; no imitation of such sites; keep a legitimate labour-management aesthetic: strong hierarchy, polished cards, clear CTAs, smooth transitions, good discovery, personalization within role.

08-NATIVE-ANDROID/ANDROID-PERMISSIONS.md

Android Permissions (current + native)

Status: current (VERIFIED) + native (PROPOSED). Commit audited `2dd2fe4`.

Current permissions (VERIFIED)

- Main `AndroidManifest.xml` declares **no** **`<uses-permission>`** — not even `INTERNET`.
- File backup/restore uses SAF via `file_picker` → **no storage permission required**.
- Result: no runtime permission requests, no permission-grant flow, no permanent-denial handling needed today.

Native permissions strategy (PROPOSED)

Principle: **never request permissions preemptively**; request contextually when the feature needs them; explain why; handle denial & permanent denial with settings recovery.

Permission	Feature needs it	When requested	Rationale / security note
INTERNET	Firebase Auth/Firestore/FCM	runtime capability (normal)	required once backend added
POST_NOTIFICATIONS (Android 13+)	FCM push (owner announcements, driver job notifs)	on enabling notifications, contextually	explain; graceful if denied (in-app notification centre)
ACCESS_FINE_LOCATION / COARSE (evaluated)	only IF driver trip location/ geofence tracking is a product need	on first driver location task	PROPOSED & conditional — do not add unless justified; alternatives (manual status) preferred for v1
CAMERA / gallery read	capture job-site/work images (PROPOSED)	on first capture	use photo picker (no permission) where possible

Permission	Feature needs it	When requested	Rationale / security note
Media/Storage	none needed if using SAF + photo picker	—	avoid broad storage permissions
Foreground service / exact alarm (evaluated)	scheduled reminders/ background sync	only if needed	keep optional

Each permission when added must document

Why it exists · which feature needs it · when requested · user explanation · denial behaviour · permanent-denial behaviour · settings recovery · security implications. (Native ADD will maintain this register.)

Current conclusion

No permissions to audit for the current offline app beyond manifest absence (VERIFIED, good). Native permissions only as features require.

08-NATIVE-ANDROID/IMPLEMENTATION-ROADMAP.md

Native Android Implementation Roadmap (proposed)

Status: PROPOSED sequence. No native work implemented. Commit audited `2dd2fe4`.

Guiding principle

Rebuild as a **secure, offline-first, role-based** product that preserves validated behaviour, fixing documented defects. Gate releases on `06-QUALITY/PRE-RELEASE.md`.

Phases

Phase 0 — Decision & foundation

- Resolve open questions (OQ-1..OQ-8): product scope, single vs multi-org, owner identity/brand, offline-first requirement, driver/labourer login need, audit/retention, release target.
- Decide package id (resolve `roshan` vs intended), project skeleton, DI (Hilt), design system (ANDROID-DESIGN-SYSTEM.md), CI.
- **Security prerequisite:** rotate/purge committed keystore file before anything ships.

Phase 1 — Offline core (no backend yet)

Port validated domain offline: local Room + DataStore; trip/work/attendance CRUD; trip numbering; search; history; backup/restore; Compose UI owner-only; tests.

Phase 2 — Authentication & roles

Firebase Auth; provisioning owner via Cloud Function; role model; server-authoritative rules; login/session; role-based navigation.

Phase 3 — Backend data & sync

Firestore schema + security rules + emulator tests; offline queue & reconcile; Cloud Functions for privileged/transactional ops; audit logging.

Phase 4 — Driver & (optional) Labourer journeys

Role-scoped apps/jobs; job assignment & status; notifications (FCM + rules + deep links); attendance recording/confirmation.

Phase 5 — Reports, backup/DR, monitoring

Reports/exports; authenticated cloud backup + retention; Crashlytics + monitoring + alerting; admin/audit tools.

Phase 6 — Release hardening

Accessibility & localization pass; macrobenchmarks & performance; release gate; staged rollout; post-release monitoring.

Cross-cutting (throughout)

- Tests accompany every phase (unit/ViewModel/repository/UI/Firebase-emulator/RBAC/security/accessibility/perf).
- Keep docs truthful and in sync (AGENT.md).
- No fake production data/backend; mocks only in tests.

Exit criteria per phase

Each phase ends green on its test set + no CRITICAL/HIGH open security issues + docs updated.

Effort/migration complexity

- Domain data logic is well-scoped → low-medium.
- Secure multi-user backend + roles is new → medium-high.
- Preserving offline-first while adding sync requires deliberate conflict design → medium-high. See `FLUTTER-TO-ANDROID-MIGRATION-MATRIX.md`.

FLUTTER->ANDROID MIGRATION MATRIX

08-NATIVE-ANDROID/FLUTTER-TO-ANDROID-MIGRATION-MATRIX.md

Flutter → Native Android Migration Matrix

Purpose: comprehensive mapping of every current Flutter implementation element to a native Android equivalent. Status legend per `00-AUDIT-INDEX.md`. Commit audited `2dd2fe4`.

Tech/dependency mapping

Flutter element	Current version	Native Android equivalent	Notes
Dart / sealed classes	SDK ^3.11.0	Kotlin sealed classes / data classes	
Flutter widgets/ Material3	dark theme	Jetpack Compose + Material 3	rebuild UI
flutter_bloc	9.1.1	ViewModel + StateFlow (UDF/MVI)	state layer changes
go_router	17.2.3	Navigation Compose / Type-safe nav	replace `extra`-based passing w/ SavedStateHandle
get_it	9.2.1	Hilt (Dagger)	
dartz Either	0.10.1	Kotlin Result / sealed error	map Failure types
equatable	2.0.8	data classes	
Hive	2.2.3	Room (relational) or DataStore	offline store
hive_generator / .g.dart	2.0.1	Room annotation processor	
uuid	4.5.3	UUID / server ids	
intl	0.20.2	java.time + DateTimeFormatter / Android strings	localization
flutter_screenutil	5.9.3	Compose adaptive layouts	drop fixed 360x690
file_picker	8.3.7	SAF (ACTION_CREATE_DOCUMENT / OPEN_DOCUMENT) + Photo Picker	backup
google_fonts	8.1.0	bundled font resources	avoid runtime fetch
flutter_animate	4.5.2	Compose animation APIs + Material motion	reduced-motion aware
glassmorphism_ui	0.3.0	M3 tonal surfaces (re-evaluate glass)	perf/design
path_provider	2.1.5	context.filesDir / Room	
cupertino_icons	1.0.9	Material icons	
(none)	—	Coil (images)	new
(none)	—	WorkManager (background)	new
(none)	—	Firebase SDKs	new (see Firebase docs)

Build/platform mapping

Flutter	Native
`pubspec.yaml` version 1.0.0+1	versionCode/versionName via Gradle
`android/app/build.gradle.kts` (AGP 8.11.1, Kotlin 2.2.20, Gradle 8.14)	Gradle Kotlin DSL (keep/upgrade)
minSdk 24	keep/raise (Firebase min)
compileSdk/targetSdk = Flutter defaults	pin explicitly
release keystore via key.properties	Play App Signing + CI secret store
proguard-rules.pro	R8/proguard with Compose keep rules
AndroidManifest (no permissions)	add only required; runtime requests contextual
`.metadata` Flutter revision	n/a

Source-file mapping (areas)

Flutter area	Purpose	Native area
`lib/core/database/hive_setup.dart`	box open/adapters	`:data:local` Room db + migrations
`lib/core/error/failures.dart`	typed failures	sealed `Failure` sealed class
`lib/core/usecases/`	usecase base	domain `UseCase` / usecase classes
`lib/features/work/domain/**`	entities/usecases/repo iface	`:domain`
`lib/features/work/data/**`	repo impl + data source + models	`:data:local` / `:data:remote` + mappers
`lib/features/work/presentation/bloc/**`	events/states/bloc	ViewModel + UiState + sealed UiEvent
`lib/features/*/presentation/**` screens	screens	Compose screens per role
`lib/shared/**` layout+widgets	shared components	`:core:designsystem` composables
`lib/routes/app_router.dart`	routes	Navigation graph
`lib/config/di`	DI	Hilt modules
`lib/theme`	theme	Compose theme
`lib/main.dart`	boot	`Application` + MainActivity + DI init

Screen parity (see SCREENS.md S-IDs)

Splash→Compose splash; Dashboard→Owner Home; History→History; Analytics→Reports; Settings→Settings+Account; Trip Details→Job/Trip detail; Add/Edit Work→Job/Trip editor; Confirm Next Trip→Copy-next-trip confirm. `/details` orphan → remove or make reachable day detail.

Migration risk summary by area

Area	Complexity	Risk	Notes
Data/domain rules	Low	Low-Med	well-scoped; re-implement + test trip numbering
UI rewrite	High	Med	UX redesign opportunity; reuse specs
Offline store	Med	Med	Hive→Room; schema version/migrate
Auth/RBAC/Firebase	High	High	net-new; secure-by-design
Notifications/sync	High	Med-High	new; offline-first tension
Backup/DR	Med	Med	make encrypted/signed; WorkManager

Full feature parity table

See `FEATURE-PARITY-MATRIX.md` (every feature F-01..F-29).

08-NATIVE-ANDROID/FEATURE-PARITY-MATRIX.md

Feature Parity Matrix (current → native)

Purpose: ensure the native rebuild retains every current feature (or explicitly drops it). Status: current = VERIFIED; native column = PROPOSED. Commit audited `2dd2fe4`. Feature F-IDs from `01-PRODUCT/FEATURE-CATALOG.md`.

F-ID	Current feature	Current status	Native Android equivalent	Firebase dependency	UX change	Migration risk	Priority
F-01	Splash → dashboard	VERIFIED WORKING	Compose splash/ session gate	none	minor	low	high
F-02	Today dashboard summary	VERIFIED WORKING	Owner Home composable	optional	low	low	high
F-03	Pull-to-refresh	VERIFIED WORKING	Compose pull-refresh on server lists	when online	low	low	medium
F-04	Dashboard search	VERIFIED WORKING	Search on home/ history	none/query	medium (extend to history)	low	medium
F-05	Dashboard filter	PARTIAL (state-only)	Full filter bottom-sheet	none/query	high (implement)	low	medium
F-06	Add Work+Trip form	VERIFIED WORKING	Job/Trip add/edit composable	store via Firestore	low	medium	high
F-07	Next-trip quick add	VERIFIED WORKING	Copy-last-trip action	store	low	medium	high
F-08	Auto trip numbering	VERIFIED WORKING	Domain usecase/ server function	CF for server-authoritative numbering	none	medium	high
F-09				yes	none	medium	high

F-ID	Current feature	Current status	Native Android equivalent	Firebase dependency	UX change	Migration risk	Priority
	Edit trip/work preserving id/date	VERIFIED WORKING	Edit composable + Room/Firestore upsert				
F-10	Labour master list	VERIFIED WORKING	Crew/Labourer list	users/crew doc	low	low	high
F-11	Labour add/edit/ remove in trip	VERIFIED WORKING	Attendance crew editor	yes	low	medium	high
F-12	Attendance toggle	VERIFIED WORKING	Attendance toggles (role-aware)	attendance doc	low	medium	high
F-13	Delete latest trip	VERIFIED WORKING	Remove trip (undo, role-gated)	yes	low	low	medium
F-14	Delete specific trip (cascade)	VERIFIED WORKING	Cascade delete server-managed	yes	none	medium	high
F-15	History grouped	VERIFIED WORKING	History list/paging	Firestore query	medium	low	medium
F-16	Analytics KPIs	VERIFIED WORKING	Reports/KPI screens	aggregation (CF/scheduled)	medium	medium	medium
F-17	Backup `.labourbackup`	VERIFIED WORKING	Signed/encrypted export + cloud backup	Storage	medium (auth)	medium	medium
F-18	Restore w/ rollback	VERIFIED WORKING	Restore service (WorkManager)	Storage/CF	low	medium	medium
F-19	Draft autosave	VERIFIED WORKING	Draft store (DataStore/Room)	none	low	low	high
F-20	Theme	PARTIAL (dark-only)	Light/dark/ dynamic tokenized theme	none	high (add light/ dynamic)	low	medium
F-21	About/Theme tiles	BROKEN/ STUB (inert)	Real About + settings	none	high (implement)	low	low
F-22	Details screen	PARTIAL/ orphaned	Day-detail screen (reachable) or remove	yes	medium	low	medium
F-23	Auth/accounts	MISSING	Firebase Auth	**yes**	high (new)	high	high
F-24	Roles/RBAC	MISSING	Backend roles + rules	**yes**	high (new)	high	high
F-25	Notifications	MISSING	FCM + CF role-scoped	**yes**	high (new)	medium	medium
F-26	Cloud/backend/ sync	MISSING	Firestore + offline reconcile	**yes**	high (new)	high	high
F-27	Payments/reports service	MISSING	Reports first; payments only if justified	**yes**	high (new)	high	optional
F-28	Accessibility	MISSING/ partial	Built-in Compose semantics	none	high	low	high

F-ID	Current feature	Current status	Native Android equivalent	Firebase dependency	UX change	Migration risk	Priority
F-29	Localization	MISSING	strings.xml/ plurals/RTL-ready	none	medium	low	medium

Decision callouts

- Core offline domain features (F-01..F-19) map cleanly → preserve; priority high.
- F-05/F-22 were partial → implement or remove intentionally in native.
- F-23..F-27 are net-new and drive the Firebase architecture; not present today.
- Offline-first must be preserved even as F-26 adds backend.

06-QUALITY/BUG-DEFECT-REGISTER.md

Bug & Defect Register

Status: VERIFIED findings (static). Many require device execution to fully reproduce; marked UNVERIFIED where runtime-dependent. Commit `2dd2fe4`. Severity: CRITICAL/HIGH/MEDIUM/LOW/INFORMATIONAL.

Schema fields per row where relevant: ID, Sev, Priority, Area, File, Symbol, Repro, Observed, Expected, Root cause, Evidence, Impact, Security impact, Recommended fix, Status.

ID	Sev	Priority	Area	File / Symbol	Observed	Expected	Root c
BUG-01	CRITICAL	1	Security	`android/app/keystore.jks.bak` (tracked, all tags)	Private-key/keystore-like DER file committed	No key material in repo	`.giti` misses `*.jks` force-a commit 55b214
BUG-02	MED	2	UX	`details_screen.dart` search `onChanged:(){}` + filter `onPressed:(){}`	Search & filter icons inert	Should filter	Incomp screen; orphan `detail`
BUG-03	MED	2	UX	`details_screen.dart` Dismissible	Both backgrounds show blue edit icon; swipe maps endToStart→delete confirm, else→edit	Swipe affordance should match action	Copy-p backgr
BUG-04	MED	2	State	`work_bloc.dart` `_onFilterDashboard` / load path	Filter state stored; `matchesFilter` always true; no UI dispatches filter	Filtering works	Filter unimpl + not s
BUG-05	MED	2	State	`work_bloc.dart` `_onAddQuickTrip`	Empty handler (comment says UI navigates)	Remove dead handler	Incomp refacto
BUG-06	LOW	3	UX	`empty_state.dart` + history/ analytics empty	EmptyStateWidget always draws a button; callers pass `ctaText:''`	No blank button	Shared assume
BUG-07	LOW	3	Settings	`settings_screen.dart` Theme Mode / About tiles `onTap:(){}`	Inert tiles shown as interactive	Implement or hide	Stubs
BUG-08	MED	2	Consistency	`date_time_utils.dart` vs root `PRD.md` §36	Code: 04:00–11:59 Morning; PRD text: 00:00–11:59	Aligned spec	Doc dri
BUG-09	MED	2	Trip Details	`trip_details_screen.dart` edit action	Edit only works when state is `TripDetailsLoaded`; otherwise silently no-ops	Robust edit	Guard on tran state

ID	Sev	Priority	Area	File / Symbol	Observed	Expected	Root c
BUG-10	MED	2	Labour edit	`work_bloc.dart` `_onSaveLabour`	Emits WorkActionSuccess, does not reload open trip list; labour name edit may not refresh list	Refresh list	Comme acknow
BUG-11	MED	2	Duplicate switch	`trip_details_screen.dart`	Duplicate/overlapping switch branches (dead)	Single exhaustive switch	Merge
BUG-12	LOW	3	Hygiene	`work_local_data_source.dart` , `work_repository_impl.dart` duplicate `@override`	Duplicate annotations	Single	Merge
BUG-13	MED	2	Restore	`settings_screen.dart` restore	Row-count verification compares length only; no semantic/field validation on injected data	Authenticated/ validated restore	No sigr
BUG-14	LOW	3	Error UX	Dashboard/Trip error states	Error text w/o retry path	Recovery path	Missing
BUG-15	MED	2	Security/PII	Backup + Hive plaintext	PII unencrypted	Encrypted	No cryp
BUG-16	LOW	3	Data model	`Labour.phoneOptional`	Field unused by UI	Confirm intent or remove	Half-ad
BUG-17	LOW	3	Nav robustness	`app_router.dart` `state.extra as Trip`	Hard cast; deep-link without extra would throw	Safe nav	Extra-b nav
BUG-18	INFO	4	Dead code	`ValidationFailure` unused; `AddQuickTripEvent` empty; Details orphan route	Dead code present	Minimal dead code	Incomp feature
BUG-19	LOW	3	Analytics/ History empty	Empty-state blank CTA & HistoryEmpty grouping	Clean empty UI	Shared comp assumption	Blank b

Notes

- No defects classified as a confirmed runtime crash without device execution; runtime reproduction of BUG-03/09/17 is `UNVERIFIED` here.
- Full native feature gaps (auth/RBAC/notifications/etc.) are tracked as PROPOSED features, not defects of the current offline product.

Status legend

OPEN = needs fix (no source changes made during this audit-only phase).

06-QUALITY/QUALITY-RISKS.md

Quality Risks

Status: VERIFIED current; PROPOSED mitigations. Commit `2dd2fe4`.

ID	Risk	Likelihood	Impact	Mitigation
QR-1	Signing-key exposure from committed keystore backup	MED (already present)	HIGH	Rotate; purge; secret-scan CI
QR-2	Data loss via silent partial-write in full-trip save (not transactional)	LOW-MED	HIGH	Wrap saves in batch/transaction; verify counts
QR-3	No schema migration → future model change breaks Hive	MED (long-term)	HIGH	Version & migrate; add migration tests
QR-4	Unauthenticated single-user assumption breaks if product opened up	HIGH (if scope changes)	HIGH	Native adds auth/RBAC from the start
QR-5	Backup plaintext PII + no integrity	MED	MED	Encrypt/authenticate
QR-6	Runtime execution/test pass not reproduced in this audit	MED	MED	Re-run suite in a Flutter-enabled CI before claiming readiness
QR-7	Dead/inert code and misleading swipe affordance erode UX confidence	MED	LOW-MED	Clean up (safe, cheap)
QR-8	Duplicate definitions risk merge conflicts/ reintroduction	LOW-MED	MED	Single shared models; automated checks
QR-9	No crash/telemetry (offline) → invisible issues	MED	MED	Native Crashlytics; manual QA now
QR-10	Pre-existing docs may over-claim production readiness	MED	MED	Reconcile docs with verification (this suite)

Overall

The product is functionally coherent for a single-user offline tool but carries one critical security/ops issue (keystore), several data-integrity/migration risks, and a testing-execution verification gap.

Verification status

Risks VERIFIED statically. Probabilities are qualitative judgments (not measured).

06-QUALITY/TECHNICAL-DEBT.md

Technical Debt Register

Status: VERIFIED. Commit `2dd2fe4`. Only substantive debt listed (not style preferences).

Debt by category

Category	Item	Ref
Architectural	Presentation widgets reach into Hive (dashboard labour lookup; add-edit draft) breaking layering	`dashboard_screen.dart`, `add_edit_work_screen.dart`
Architectural	Business aggregation in screens (history grouping, analytics KPIs)	history/analytics
Architectural	Backup/restore data logic embedded in a settings screen	`settings_screen.dart`
Architectural	Navigation-as-state via `NavigateToConfirmNextTripState` + multi-emit	`work_bloc.dart`
Architectural	Filtering stored in bloc private mutable fields, not state	`work_bloc.dart`
Code	Duplicate `LabourFormModel` top-level class in two screens	both create screens
Code	Duplicate `@override` annotations; duplicate/overlapping switch cases	data source, repo impl, trip_details
Code	Empty handlers / dead events (`AddQuickTripEvent`), dead `ValidationFailure`, orphaned `/details`	bloc, failures, router
Dependency	`google_fonts` runtime network fetch vs offline-first	app_theme
Persistence	No schema migration framework; JSON-in-string draft; deterministic work-id semantics ambiguity	DATABASE/STORAGE
Persistence	Hive plaintext; no compaction/retention	
UI	Two overlapping "add trip" entry flows (Add Work vs Next Trip) create conceptual overlap	add_edit vs confirm_next_trip
UI	Dark-only, non-tokenized styles, fixed 360×690 design	theme/screenutil
Testing	`restore_logic_test` tests rules indirectly; missing tests for settings/details/draft/confirm screens	TEST-COVERAGE-MAP
Security	Committed keystore-like file (item treated as bug/security debt BUG-01)	SECRETS-AUDIT
Infra	No CI/CD, no automated quality gate in repo	07-OPERATIONS
Docs	Pre-existing agent docs may over-claim execution/readiness vs current verification	CODEBASE-MAP note

Prioritisation suggestion

1. Security debt (BUG-01, encryption, backup integrity).
2. Data-integrity & migration readiness (schema versioning, semantics of deterministic work id).
3. Code-debt cleanup (duplicates/dead code/inert UI) — small, safe, high value.
4. Layering refactors — do with the native rebuild rather than risky mid-life Flutter churn.
5. CI/test & documentation truth alignment.

Verification status

VERIFIED entries. Some are PROPOSED improvements rather than blocking defects.

Pre-Release Quality Gate

Purpose: a release must NOT be declared production-ready until the gate below passes. Status: this is the required gate (applies to native and any Flutter release). Commit `2dd2fe4`.

Gate checklist

- [ ] Build succeeds (debug + release).
- [ ] `flutter analyze` / static analysis clean (or lint gate passes).
- [ ] Unit tests pass.
- [ ] Widget/integration tests pass.
- [ ] Lint passes.
- [ ] Security checks pass (secret scan; no secrets/keys in repo/APK).
- [ ] Critical flows pass (create trip → attendance → history/analytics → backup/restore).
- [ ] Authentication works (native) / N/A for current offline app.
- [ ] RBAC works (native) / N/A current.
- [ ] Backend/Firebase authorization works (native) / N/A current.
- [ ] Notifications work (native) / N/A current.
- [ ] Offline behaviour acceptable (current = inherently offline).
- [ ] Error states work (add retry where missing).
- [ ] Accessibility checks pass (address HIGH A-1/A-2).
- [ ] Performance acceptable (measure cold start/scroll).
- [ ] No known CRITICAL/HIGH vulnerabilities remain (resolve BUG-01 + keystore before any release).
- [ ] No production secrets exist in the APK/repository (rotate + purge keystore first).
- [ ] Release signing correct (keystore in CI secrets, not repo).
- [ ] Monitoring active (native: Crashlytics) / N/A offline current.
- [ ] Crash reporting works (native).
- [ ] Database rules reviewed (native).
- [ ] Backup/recovery strategy exists & tested (current: manual `.labourbackup` verified).
- [ ] Release artifact verified (install & smoke on clean device).

Current-product verdict vs gate

Criterion	Verdict
The code appears release-shaped (prior RC tags)	TRUE (structural)
Independent execution of the gate in this environment	**UNVERIFIED** (no Flutter SDK)
Secret hygiene	**FAILS** (BUG-01 keystore present)
Production readiness claim	Deferred pending gate execution + secret remediation

Native

Apply identical gate plus Firebase rules review, App Check, audit logs, and macrobenchmarks before each release.

FINAL AUDIT CONCLUSION

00-AUDIT-INDEX.md

Audit Index & Executive Control Document

Purpose: single executive roll-up of the forensic audit. Every claim is classified. Commit `2dd2fe4`. Audit date 2026-09-07.

1. Repository snapshot

Field	Value
Repository	`https://github.com/Manash07Bhoi/LABOUR-PARTY-2.git`
Commit audited	`2dd2fe4ed85e9f0e2a420e3a383fed1e75b8a21b`
Branch (audited HEAD)	`main` (remote `origin/main`); audit on `audit/documentation`
Tags	`RC-1.3`, `v1.0.0`, `v1.0.0-rc1`, `v1.0.1-hotfix-rc1`
Working tree	clean at clone
Contributors (git)	google-labs-jules[bot] (64), Manash07Bhoi (48); 112 commits
Dart files (lib)	47 (incl. 5 generated `.g.dart`); 6,140 lines
Test files	27; ~2,505 lines
Backend services	NONE (offline app)
Build config	AGP 8.11.1, Kotlin 2.2.20, Gradle 8.14, Java 17
Android	namespace/appid `com.roshan.labourparty`; minSdk 24; compile/target via Flutter defaults; version 1.0.0+1
Firebase	NONE present
CI/CD	NONE present

2. Audit environment & execution

- Environment: sandbox without a Flutter/Dart SDK (git, Python, Node, Java 11 present).
- Execution attempted: **not possible** — no Flutter SDK to install/build/test; no Android emulator. Audit is **static + evidence-based**.
- Runtime claims in prior repo docs (screenshots under `docs/screenshots`) were **not independently reproduced**. All runtime-dependent statements are `UNVERIFIED`.

3. Application status summary

Metric	Value	Classification
Product	Offline-first single-user labour/trip manager	VERIFIED
Screens (routes)	9 routed screens + splash (S-01..S-09)	VERIFIED

Metric	Value	Classification
Bottom-nav destinations	4 (Dashboard/History/Analytics/Settings)	VERIFIED
Features catalogued	29 (F-01..F-29); 17 working, 5 partial/stub, 7 missing/proposed	VERIFIED
Roles	0 (single user)	VERIFIED
Auth	None	VERIFIED
Backend	None	VERIFIED
Dependencies	flutter_bloc/go_router/hive/get_it/equatable/dartz/uuid/intl/screenutil/file_picker/google_fonts/flutter_animate/glassmorphism_ui/path_provider (runtime); build_runner/hive_generator/mocktail/flutter_lints (dev)	VERIFIED
Local data	Hive boxes: work/trip/labour/trip_labour/draft	VERIFIED

4. Critical / high / medium findings

Sev	ID	Finding
CRITICAL	SEC-1 / BUG-01 / SA-1	`android/app/keystore.jks.bak` (DER/PKCS#8 v0 private-key structure) committed on main + all 4 tags + ~26/31 remote branches; `.gitignore` misses `*.jks.bak`. Rotate & purge required.
HIGH	A-1/A-2	Accessibility: icon buttons untagged; low-alpha text contrast risk (native & current).
MEDIUM	BUG-02..05,09..13	Inert search/filter & orphan `/details`; filter unimplemented; misleading swipe; data-integrity/restore-auth; labour edit refresh; duplicate code.
MEDIUM	ST-1/2/15	PII at rest & backups unencrypted; backup unsigned.
MEDIUM	T-1	Test/execution not reproduced (no SDK in env).
LOW	BUG-06..08,14,16..19	Blank empty-state CTA; inert settings tiles; session-boundary doc drift; no retry; dead code; unused phone field.

5. Ratings (qualitative, evidence-based — NOT fabricated numeric scores)

Dimension	Rating	Basis
PRODUCT MATURITY	MODERATE (single-user core is coherent; multi-role/backend absent)	verified scope
ENGINEERING MATURITY	MODERATE (clean-ish layering but presentation leaks, dead code, no CI/migration framework)	code audit
SECURITY MATURITY	WEAK-FOR-MULTIUSER / LOW-RISK-TODAY; one CRITICAL config issue (keystore)	SECURITY-AUDIT
UX MATURITY	MODERATE (polished core; inert/dead paths; accessibility gaps)	UX-AUDIT
TESTING MATURITY	MODERATE (good test inventory but execution unverified + coverage gaps)	TESTING-AUDIT
PERFORMANCE MATURITY	UNVERIFIED (structural design good; no runtime measurement)	PERFORMANCE-AUDIT
PRODUCTION READINESS	**NOT READY / UNVERIFIED** — secret-hygiene FAIL (keystore) blocks any release; runtime gate not run	PRE-RELEASE

Dimension	Rating	Basis
NATIVE MIGRATION READINESS	READY TO SPEC, core domain low-risk; auth/backend/sync are net-new	FEATURE-PARITY-MATRIX

6. Documentation index

See `DOCUMENTATION-INDEX.md` (full tree). Required docs verified present (programmatic check in `verify` section).

7. Migration & production readiness statements

- **Native migration readiness:** Core offline domain (F-01..F-19) maps cleanly and is low-medium risk to rebuild. The secure multi-user backend, roles, notifications and sync (F-23..F-27) are new high-risk scope. Start with Phase 0 decisions + keystore remediation (IMPLEMENTATION-ROADMAP.md).
- **Production readiness (current Flutter):** NOT READY for an actual public release until (1) keystore file removed/rotated, (2) the release gate is executed on a Flutter-enabled machine, (3) data-integrity/migration tests are re-run. Functionally coherent, but readiness is UNVERIFIED in this environment.

8. Unresolved questions (UNVERIFIED)

OQ-1 operator count · OQ-2 owner identity/brand ("Ramesh Sahu" vs package "roshan") · OQ-3 cloud vs offline-only future · OQ-4 exact session boundary (code vs PRD text) · OQ-5 soft vs hard delete future · OQ-6 audit/retention depth · OQ-7 driver/labourer login scope · OQ-8 audit granularity. Runtime numeric validation for many claims is UNVERIFIED.

9. Overall audit conclusion

The existing product is a **genuinely useful, coherent offline-first single-user labour/trip tracker** with a well-scoped domain and good structural intent. It is **not** a multi-user/RBAC product, **not** connected to any backend, and it carries **one critical repository-hygiene/security issue** (committed keystore-like file). The documentation here separates current reality from proposals so the native rebuild can preserve validated behaviour and deliberately fix the weak areas.

Final Audit Report

Repository

- Commit audited: `2dd2fe4ed85e9f0e2a420e3a383fed1e75b8a21b`
- Branch: `main` (work on `audit/documentation`)
- Build status: NOT BUILDABLE in this environment (no Flutter SDK). Static structure is release-shaped; execution UNVERIFIED.

Product

- Purpose: offline-first labour/trip management (Work→Trip→Labour+attendance), history, analytics, backup/restore.
- Users: single operator. Roles: none.
- Major features: F-01..F-19 working core; F-20..22 partial; F-23..29 absent/proposed.

UX

- Screens: S-01..S-09 (9 routed + splash). Navigation: splash→shell(4 tabs); pushed detail/edit/confirm screens.
- Major UX problems: orphan `/details`, inert search/filter, misleading swipe delete, no retry, search only today, accessibility, dark-only.
- Opportunities: role IA, day detail, history search/filter, bottom-sheet filters, light/dynamic theme.

Engineering

- Architecture: Flutter + flutter_bloc + get_it + go_router + Hive + dartz; Clean-ish layering w/ presentation leaks.
- Dependencies/data/APIs: listed above; no remote API; Hive schema; BLoC state; type failures.

Security

- Critical: committed keystore-like `.jks.bak` (main+tags+branches).
- High: (accessibility aside) none; current runtime attack surface minimal offline.
- Medium: PII at rest/backup unencrypted; backup unsigned; restore trusts content.
- Low: raw-exception text; duplicate-code merge hygiene; deep-cast nav.

Performance

- Measured: none re-run. Suspected risks: full-box scans; labour name lookups; backup memory; fonts network fetch.
- Unverified: all numeric (cold start, jank, benchmarks).

Testing

- Existing: 27 files (unit/widget/integration/perf/e2e + mock helper). Missing: execution confirmation; settings/details/draft/confirm UI tests; accessibility/security/locale tests.
- Execution results: UNVERIFIED (no SDK).

Firebase

- Existing: none. Recommended: Auth/Firestore/Storage/CF/AppCheck/Crashlytics (+FCM, optional Analytics/Remote Config), server-authoritative rules & audit (08-NATIVE-ANDROID).

Native Android

- Recommended architecture: Kotlin+Compose+M3+Navigation+ViewModel+StateFlow+CleanArch+Hilt+Coroutines+Repository+Room/DataStore/Coil/WorkManager+Firebase.
- Migration complexity: domain low-medium; UI medium; auth/backend/sync high.
- Redesign opportunities: role IA, day detail, filters, accessibility, theme, backup/DR.

Production readiness

NOT READY for public release (secret hygiene fails; release gate unexecuted). Functionally coherent but readiness **UNVERIFIED** here. Recommend: remediate keystore, run the gate on a Flutter-enabled CI, then reassess.

Native Android migration readiness

READY TO SPEC / LOW-MED CORE RISK — proceed to Phase 0 (decisions + keystore remediation) then native implementation.

Known unverified areas

Flutter build/test/run · benchmark numbers · runtime UX/screenshots · TalkBack/contrast numeric · CVE status of deps · whether a Play-store release exists · deeper per-branch secret history.